SeedBomb write-path change record (ch1)

SeedBomb write-path change record and throughput analysis (ch1)

Written 2026-11-12. This file records every change to bulk creation, throttling, retry, backoff and jitter, parallelism, batch sizing and the Dataverse reads around a run, from Oct 1 2026 to Nov 11 2026. Section 6 then ranks the possible causes of the throughput drop.

0. Summary

The change record

- 75 non-merge commits fall in the window. Section 4 documents 39 of them one by one: 34 that change how a run behaves, and 5 docs commits that belong to the Nov 2 to 3 work. The other 36 are listed with the reason each one does not touch the write path.
- The work came in three bursts: Oct 4 to 7 (ships as v1.0.0), Oct 28 to Nov 1 (v1.0.1, v1.0.2, master, plus three branch-only commits), and Nov 2 to 3 (branch `fix/dop-cap-at-server-hint` only). There are no commits on Oct 1 to 3, Oct 8 to 27 or Nov 4 to 11 except unmerged Dependabot bumps.
- Section 3 is a one-line-per-commit timeline. Section 2 shows the effective settings at each build side by side.

The throughput drop

Nothing here was run. This environment has no .NET SDK, no Dataverse environment and no run log, so section 6 ranks causes from the code and cannot prove one. The inputs from the owner were: the last fast build was from 9/27, more than one later build felt slow (or it is unclear which), parallelism was on automatic, and no numbers were recorded.

1. **A healthy run with automatic parallelism changed in exactly one way since 9/27.** Commit `7cb61b5` (first shipped in v1.0.1) runs every table at the server's parallelism hint. Before it, the first table, and the whole of a single-table run, ran at 8. If the environment's hint is below 8, concurrency on those tables fell by that ratio: 37% at a hint of 5, 50% at 4, 75% at 2. The SDK also reports 5 before it has seen any hint. The `[run] snapshot` log line of any v1.0.1 or later run prints the hint, so one run settles this.
2. **Every other change only costs time when the server throttles, times out or rejects rows.** In that state the builds differ a lot:
    - v1.0.0 to master: a bad batch holds one worker for minutes (SDK retries nested inside app retries, and a 4-minute timeout resent three times since `eccf60a`).
    - The branch: one throttle response stops every worker in the run for the server's `Retry-After`, a batch gets four attempts before all its rows are rejected, a data fault rejects the whole batch, and a timeout or network fault is never resent.

3. **Long runs are throttled by design.** Dataverse allows 1,200 seconds of execution time per 5 minutes, per user, per web server. Five or more always-busy requests against one web server exceed that, so a long run spends most of its life in the state where the builds differ most. This is arithmetic on the documented limit, not a measurement.
4. **Some large costs are not regressions.** Rows are generated on one thread before each table's first write, tables run one after another, and the deferred-lookup backfill sends one batch at a time while copying the target list once per row. These have not changed since before 9/27, but they are inside the summary throughput figure.

Section 7 lists the log lines that tell these apart. The shortest path to an answer is one run on the build that feels slow, with the log kept.

1. Scope, method and builds

Window. Oct 1 to Nov 11 2026: the range `a527963..a5fbbe5`, 75 non-merge commits and 14 merges. The clone was shallow at first and hid the Oct 4 commits; the full history was fetched before any analysis.

Builds.

Build	Commit	Date	Notes
Last fast build (owner)	around `16c3e75`	Sept 27	Git cannot say which commit that binary was built from. If it predates `41963f6` (Sept 26), SDK retries were still off. See section 5.
v1.0.0	`56ec5ee`	Oct 5	All of Oct 4 to 5.
v1.0.1	`78a86d2`	Oct 31	Phase 1 to 3. `PUBLISH.md` says it was withdrawn and replaced by v1.0.2.
v1.0.2	`4feba6a`	Oct 31	Same runtime code as v1.0.1; only the x86 build was dropped.
master	`b4433d2`	Nov 1	v1.0.2 plus the `createdon` rule change. Untagged.
Branch (HEAD)	`a5fbbe5`	Nov 3	`fix/dop-cap-at-server-hint`, 23 commits ahead of master, unmerged, not in any tag.

Method. Five sub-agents read the history: three wrote the per-commit record for one date window each, one modelled the batch loop at each build, and one traced everything outside the batch loop. Each read the diffs, not the commit subjects. I then checked their main claims against the code and corrected the record where needed. What I checked myself:

- The automatic-parallelism code at `16c3e75`, v1.0.2 and HEAD.
- `ThrottlePolicy.cs`, `AdaptiveBatchSize.cs` and the batch loop in `BulkCreator.cs` at HEAD, line by line.
- The `ServiceClient` construction at `cdb40f7`, `16c3e75` and HEAD, and the settings that feed it.
- The Sept 28 to 30 diffs on the write path.
- The backfill copy and the per-row currency warning.

- The SDK facts in section 2.2, against the SDK's public source.

Labels. In sections 4 and 6, "inference" marks a conclusion reasoned from the code that the code itself does not state. Line numbers are for the commit named beside them; "HEAD" is `a5fbbe5`.

2. Effective settings by build

2.1 Side by side

Setting	9/27 (`16c3e75`)	v1.0.0	v1.0.1 / v1.0.2 / master	Branch (HEAD)
Default batch size	500	500	500	500 (now a ceiling)
Batch size sent	configured, fixed	configured, fixed	configured, fixed	starts at `min(configured, 250)` for every table; ×1.5 after each call of 20 s or less; halved (floor 25) on each timeout or network fault
Batch size limit	none in code (UI box 1000)	1 to 1000	1 to 1000	1 to 1000
Automatic parallelism	8 for the first table written; the hint for later tables	same	the hint for every table; 8 only if the hint is 0 or less	same as master
Explicit parallelism	used as typed (capped at 16 from Sept 28)	1 to 16	1 to 16	1 to 16, then capped at the hint
WhoAmI warmup	automatic and 5,000 rows or more; logs the hint only	same	automatic and any rows planned	same as master
App retries per call	3 (4 attempts)	3	3	3
App backoff	`min(60 s, 2 s × 2^attempt)` plus uniform jitter of −2 s to +2 s	same	same	same; a service-protection fault waits `Retry-After` instead, with no jitter and no cap
What the app retries	`TimeoutException`, `HttpRequestException`, `IOException`, HTTP 429 as `ProtocolException`	adds any cancel the caller did not request (HttpClient timeout), from Sept 29	adds wrapped socket resets, `EndpointNotFoundException`, `ServerTooBusyException`	adds service-protection faults and HTTP 502/503; for creates, timeouts and network faults are no longer resent
SDK `MaxRetryCount`	not set, so 10	not set, so 10	not set, so 10	0 (Settings, range 0 to 10)

Setting	9/27 (`16c3e75`)	v1.0.0	v1.0.1 / v1.0.2 / master	Branch (HEAD)
SDK `RetryPauseTime`	not set, so 5 s	not set, so 5 s	not set, so 5 s	0 s (Settings, range 0 to 300)
HTTP timeout	SDK default, 4 min	4 min	4 min	4 min (Settings, range 1 to 30)
Service-protection fault	SDK waits `Retry-After` inside the call, up to 10 times; then terminal in the app	same	same	app waits `Retry-After`; every worker in the run pauses; up to 3 retries
`ExecuteMultiple` fallback	on any fault from `CreateMultiple`	same	same	only when `CreateMultiple` is unsupported (`0x80040800`)
Timeout on a create	`TimeoutException` resent 3 times; an HttpClient cancel ends the run as cancelled	both resent 3 times	both resent 3 times	not resent; a keyed table splits into halves, an unkeyed table reports the rows as outcome unknown
Pre-run lookup reads	every `lookupRandom` target, up to 1,000 rows	adds implicit required lookups	same as v1.0.0	implicit lookups to in-run parents read nothing
Per-batch log line	none (Debug only)	none	one `[timing] batch` line	same
`Dataverse.Client` package	1.2.27	1.2.27	1.2.27	1.2.27

Unchanged in every build: one shared `ServiceClient` (`useUniqueInstance: true`, `EnableAffinityCookie = false`, never cloned), no connection-limit or thread-pool settings, an MSAL silent token per request, progress posted to the UI at most every 100 ms, tables written strictly one after another, and all rows of a table generated on one thread before its first write.

File and line references for each cell are in the parameter snapshots at the end of windows A, B and C in section 4.

2.2 SDK facts checked in the public source

Several entries below rest on defaults inside `Microsoft.PowerPlatform.Dataverse.Client`. The earlier investigation (`sb-investigate.md`) stated them and the sub-agents could not confirm them. I read the SDK's public source (`microsoft/PowerPlatform-DataverseServiceClient`, master branch) and they hold. The repository pins 1.2.27 and that tag was not checked separately.

Fact	Where in the SDK source
`RecommendedDegreesOfParallelism` starts at **5** and is replaced whenever a response carries `x-ms-dop-hint`	`ConnectionService.cs:675`, `:2414-2419`
`MaxRetryCount` defaults to 10 and `RetryPauseTime` to 5 s	`ConfigurationOptions.cs:41`, `:53`

Fact	Where in the SDK source
`MaxConnectionTimeout` defaults to 4 minutes	`ConnectionService.cs:220`
The SDK retries the three service-protection fault codes using `ErrorDetails["Retry-After"]` when present, else `RetryPauseTime + 2^n` s; also 502, 503 and SQL transient errors	`ServiceClient.cs` `ShouldRetry`
With `MaxRetryCount = 0` the SDK retries nothing and rethrows the original exception object, so the app does see the `FaultException<OrganizationServiceFault>` with its `Retry-After` detail	`ShouldRetry` first check; `ExecuteAsync` ends with `throw LastException`
`LastException` is stored on the client's logger, one per `ServiceClient` instance. With one shared client and several calls failing at once, a caller can be handed another caller's exception. Microsoft's parallel sample avoids this with `Clone()` per worker.	`ServiceClient.cs:433`, `DataverseTraceLogger.cs:192`
The async execute path takes no lock, so a shared client does not serialise parallel async calls	`Command_ExecuteAsyncImpl`

Two consequences for this repository:

- The `hint <= 0` branch of `ResolveDop`, which falls back to 8, cannot be reached with a real `ServiceClient`. An environment that sends no hint runs at 5 from v1.0.1 on, where the 9/27 build ran at 8.
- The shared `LastException` is a risk for the branch more than for master, because the branch decides what to do from the exact exception type (wait, split, or report unknown). It is a risk read from the SDK source, not something observed.

3. Timeline at a glance

One row per commit that changes run behaviour. "Healthy" means no throttling, timeouts or faults. Full entries are in section 4.

Date	Commit	First in	What changed	Healthy run	Throttled or faulty run
Oct 4	`7488e3a`	v1.0.0	`Cryptography.Xml` pinned to 10.0.10 for `net10.0` projects; audit advisories fail the build	neutral	neutral
Oct 4	`69dcfb2`	v1.0.0	`GenerationLimits` owns the bounds (batch 1 to 1000, parallelism 1 to 16, 100,000 rows) and `BulkCreator` validates them before any request	neutral	neutral
Oct 4	`a7e8b86`	v1.0.0	`RunSessionGate` held for the whole run; cached client reused only for the same profile and URL	neutral	neutral
Oct 4	`ee85321`	v1.0.0	Required-lookup check moved before the first write (no I/O); a failure after a write returns the written IDs	neutral	neutral
Oct 4	`72e80ab`	v1.0.0	Implicit `lookupRandom` rule for a required lookup whose targets are created earlier; adds up to two 500-row reads per target before the first write	first write slightly later	reads share the retry stack

Date	Commit	First in	What changed	Healthy run	Throttled or faulty run
Oct 4	`ff8fdb1`	v1.0.0	`IsTransient` flag so Retry is offered for network faults; in-run retry unchanged	neutral	neutral
Oct 4	`22a5920`	v1.0.0	Run-scoped alternate keys; retry by row index	neutral	neutral
Oct 4	`f14b561`	v1.0.0	New key scope on every Start; Retry regenerates exactly the rejected rows	raises repeat runs on keyed tables	neutral
Oct 5	`23970cf`	v1.0.0	Implicit lookups prefer parents created in the run (the reads still happen); Retry keeps the first run's topology	neutral	neutral
Oct 28	`256faa9`	v1.0.1	Wrapped socket resets retried 3 times with backoff; `Cryptography.Xml` 10.0.14	neutral	slot held about 14 s longer; possible duplicate rows
Oct 30	`695b42d`	v1.0.1	`Caching.Memory` 10.0.12 to 10.0.14	neutral	neutral
Oct 30	`35f4d23`	v1.0.1	One retry classifier; `EndpointNotFoundException` and `ServerTooBusyException` now retried; retries logged; metadata cache dispose fix; integer keys honour the column range	neutral	longer waits on a busy server
Oct 30	`d4d42ec`	v1.0.1	`[timing]` batch and phase lines and a run snapshot, always on; SDK logger and post-run count, opt-in	neutral (one synchronous log line per batch)	neutral
Oct 30	`7cb61b5`	v1.0.1	**Automatic parallelism is the server hint on every table** (was 8 for the first table); WhoAmI on every automatic run	**lower if the hint is below 8, higher if above**	higher parallelism above 8 reaches the execution-time limit sooner
Oct 30	`72fd797`, `7e95da1`	v1.0.1	Opt-in reconcile counts on `modifiedon` after a pre-run watermark read per table	neutral unless Run diagnostics is on	neutral
Nov 1	`449cac2`	master	`overriddencreatedon` written only when a rule sets it	neutral	neutral
Nov 1	`50280b4`	branch	An emptied parallelism box now means automatic (before, it most likely kept the previous number)	depends on the old number	same
Nov 1	`837d63b`	branch	Explicit parallelism above the hint is lowered to the hint	lower for an explicit value above the hint	fewer throttles
Nov 2	`1b5ef40`	branch	SDK retries set to 0; the app retries service-protection faults (waiting `Retry-After`) and HTTP 502/503	neutral	4 attempts per batch instead of the SDK's 10 inside each; calls outside `ThrottlePolicy` lose all retries
Nov 2	`e35635e`, `a2c6bcb`	branch	One run-wide `Retry-After` pause, extend-only; waiters re-check the deadline	neutral	every worker idles for the longest `Retry-After` seen

Date	Commit	First in	What changed	Healthy run	Throttled or faulty run
Nov 2	`8b50639`	branch	"Server busy, resuming in N s" on the run sheet	neutral	neutral
Nov 2	`c8a8b0f`	branch	`ExecuteMultiple` fallback only when `CreateMultiple` is unsupported	neutral	fewer requests, but one bad row rejects the whole batch
Nov 2	`5567cf6`, `0cfb873`	branch	A timed-out create is not resent: keyed tables split into halves, unkeyed tables report outcome unknown	neutral	no 4-minute resends; rows on unkeyed tables are not written
Nov 2	`938b745`	branch	Adaptive batch size from 250 rows, per table	about ±10%	smaller calls after trouble
Nov 2	`3d72627`	branch	Size halved on every timed-out call; no growth for a batch that had one	neutral	a burst of timeouts drops the size to 25 rows
Nov 2	`5dfc3aa`, `9613a9b`	branch	Implicit lookups to in-run parents skip the pre-run read; lazy fallback read if the parents wrote nothing	first write sooner	neutral
Nov 3	`1a62a2e`, `f51a23f`	branch	Repeated activity lines collapsed; one failure summary per table and message	neutral	less log noise
Nov 3	`5947b70`	branch	Settings for SDK retries (0), SDK pause (0 s) and call timeout (4 min)	neutral at the defaults	raising SDK retries brings back nested retries
Nov 3	`a5fbbe5`	branch	Network faults on a create are not resent either; same split-or-unknown path; each one halves the batch size	neutral	flaky network loses rows on unkeyed tables

Docs-only commits on the branch: `26fa1fc` (the earlier investigation), `936e754` (the plan), `aea75f5`, `120f5f5`, `194345f`.

4. Change record

The three windows below were each written from the diffs by one sub-agent and then checked. Each entry gives the area, the files, the behaviour before and after with the constants from the diff, and the effect on throughput. Each window ends with the commits that were read and found not to touch the write path, and a parameter snapshot with file and line references.

4.1 Window A: Oct 1 to Oct 7 2026 (a527963..00e6c41, ships as v1.0.0)

There are 37 non-merge commits in the range. No commit is dated Oct 1 to 3. The first is `7488e3a` on Oct 4 03:02. Four merges set the PR boundaries:

- `3671b17` (Oct 5 14:41): PR #34 `finalreview`, parents `a527963` + `23970cf`. It holds all 32 commits from `7488e3a` to `23970cf`.

- `56ec5ee` (Oct 5 15:06, tag v1.0.0): parents `ab2a40d` + `3671b17`.
- `51a9cb7` (Oct 7): PR #1 Dependabot, actions/checkout.
- `00e6c41` (Oct 7): PR #2 Dependabot, actions/setup-dotnet.

None of the merges is an evil merge. `git diff 23970cf 3671b17` is empty, and `3671b17..56ec5ee` changes only `.gitignore` (from `ab2a40d`).

Contained in v1.0.0 (`git merge-base --is-ancestor <c> 56ec5ee`): all 32 commits of PR #34 plus `ab2a40d`, which makes 33. **Landing after the tag:** `3cbf429`, `cf14ae3`, `0c01714`, `2627251` and `968f6a6`. All five are docs, solution or CI files, and none touches the write path. A sixth Dependabot commit from Oct 7, `ff6207c` ("Bump the nuget group": `System.Security.Cryptography.Xml` 10.0.10 to 10.0.12), is **not** in this range, v1.0.x, master or HEAD. It exists only on `origin/dependabot/nuget/nuget-f92cf0a58d`.

Overview of the window:

- The write core did not change: `ThrottlePolicy` retry and backoff, the `CreateMultiple` to `ExecuteMultiple` fallback, batch chunking, `Parallel.ForEachAsync` DOP resolution, and `ServiceClient` construction settings. `ThrottlePolicy` gained only a classification helper.
- What changed around the core:
  - run-size limits moved into one owner and are now enforced at the Bulk boundary;
  - a process-wide `RunSessionGate` is held for the whole run;
  - the required-lookup preflight now adds implicit `lookupRandom` rules, which pull a pre-run candidate read into runs that used to fail outright;
  - alternate keys are scoped per run;
  - Retry rebuilds exactly the rejected row indexes from the first run's full configuration.
- The `Microsoft.PowerPlatform.Dataverse.Client` version stayed at 1.2.27.
- Nothing in this window adds per-batch round trips, changes the request mix of a normal first run, or changes concurrency. On the code alone, this window is an unlikely source of a large rows-per-second drop. The candidates for second-order effects are listed under each entry.

`7488e3a` · 2026-10-04 03:02 -0400 · Pin stable Crypto.Xml for net10.0 only and stop demoting NuGet audit errors (WR-015)

- **Area:** packages
- **Files:** `Directory.Packages.props`, `Directory.Build.props` (plus 20k lines of deleted license text and notice manifests)
- **Before → After:**
  - Before: `<GlobalPackageReference Include="System.Security.Cryptography.Xml" Version="11.0.0-preview.7.26381.103" />` applied to every project, the WPF app included.
  - After: `Version="10.0.10"` with `Condition="!$(TargetFramework.Contains('-windows'))"`. Only the `net10.0` projects (Core, Bulk, tests) get the pin. The WPF app (`net10.0-windows10.0.17763.0`) now uses the Windows Desktop framework's copy.

    - The preview `System.Formats.Asn1` and `System.Security.Cryptography.Pkcs` drop out of the notice inventory, so those preview transitive versions are also gone from the app.
    - `<WarningsNotAsErrors>NU1901;NU1902;NU1903</WarningsNotAsErrors>` is removed, so audit advisories now fail the build.
    - `Microsoft.PowerPlatform.Dataverse.Client` stays at 1.2.27. No HTTP-stack package changes.
- **Throughput effect:** `neutral` (inference). `Cryptography.Xml`, `Pkcs` and `Asn1` serve XML signing and certificate parsing, not the OAuth bearer, HTTP or JSON path that `ServiceClient` uses for `CreateMultiple`. In the app, the move is from a preview build to the serviced framework build.
- **Notes:**
    - The Dependabot bump `ff6207c` to 10.0.12 sits on an unmerged branch (see the overview).
    - `GlobalPackageReference` is `PrivateAssets=all`, so the 10.0.10 pin on Bulk and Core does not flow into the WPF app.
    - The test `Load_production_manifest_has_forty_seven_components_and_four_featured` asserts that no preview component remains.

`69dcfb2` · 2026-10-04 03:51 -0400 · Give run-size bounds one owner and enforce them at load, save and the Bulk boundary (WR-014)

- **Area:** settings+limits, batch size, DOP
- **Files:**
    - `src/SeedBomb.Core/Contracts/GenerationLimits.cs` (new)
    - `GenerationConfig.cs`
    - `BulkCreator.cs`
    - `Wpf/Services/Settings/AppSettings.cs`, `JsonSettingsService.cs`
    - `Wpf/Services/Profiles/JsonProfileService.cs`
    - `GenerateViewModel.cs`, `SettingsViewModel.cs`, XAML number boxes
- **Before → After:**
    - **Constants.** `GenerationLimits` holds `MaxBatchSize = 1000`, `DefaultBatchSize = 500`, `MaxRecordCount = 100_000`, `DefaultRecordCount = 10` and `MaxDop = 16`. These replace `GenerateViewModel.MaxDop = 16`, `JsonProfileService.MaxRecordCount = 100_000` and the literals 500 and 10. Every default and maximum value is unchanged.
    - **Bulk boundary.** `BulkCreator.CreateAsync` now calls `GenerationLimits.Validate(config)` first. It throws `DataGenerationException` when `BatchSize is < 1 or > 1000`, when `MaxParallelism is < 1 or > 16` (null passes and means auto), or when any record count is above 100,000. It throws before any request is sent.
    - **Settings.** `AppSettings.Clamped()` clamps batch size to 1..1000, DOP to 0..16 and count to 1..100,000. It runs on both load and save.
    - **BuildConfig.** It now clamps `BatchSize` with `Math.Clamp(BatchSize, 1, GenerationLimits.MaxBatchSize)`. Before, `BatchSize = BatchSize` was unclamped, and only the NumberBox `Maximum="1000"` limited it.
    - **Profiles.** Save now rejects counts that the loader would reject.
- **Throughput effect:** `neutral`. The defaults are still batch 500 and DOP auto (null, which resolves to 8). A batch size above 1000 was already blocked in the UI, and Dataverse caps `ExecuteMultiple` at 1000.

- **Notes:**
  - Test `CreateAsync_BatchSizeAboveExecuteMultipleCap_ThrowsBeforeAnyRequest`.
  - `GenerationLimitsTests`.
  - `Out_of_range_settings_are_clamped_on_load`.

`a7e8b86` · 2026-10-04 10:02 -0400 · Tie run targets and Retry to a protected write-start session (CR-001)

- **Area:** gating, client config, retry-of-rejected
- **Files:**
  - `Wpf/Services/Auth/RunSessionGate.cs` (new), `ProfileAuthService.cs`, `IAuthService.cs`
  - `Wpf/Services/Dataverse/DataverseConnectionService.cs`
  - `RunViewModel.cs`, `GenerateViewModel.cs`, `ConnectionManagerViewModel.cs`, `SettingsViewModel.cs`
  - `App.xaml.cs`
  - `GenerationConfig.cs` (adds `PlannedRows`/`PlannedTotal`)
- **Before → After:**
  - **Gate.** New singleton `RunSessionGate` wraps `SemaphoreSlim(1,1)`. `RunViewModel.ExecuteCoreAsync` calls `lease = await _sessionGate.AcquireAsync(ct)` before `_generation.GenerateAsync(...)` and releases it in the outer `finally`, after `RecordRunAsync` (the history write) and `ShowLive()`. The gate is held for the full run, including the backfill and N:N phases.
  - **Other gate holders.** Startup silent sign-in, the profile switch, save and delete in `ConnectionManagerViewModel`, Settings sign-out, and `ProfileAuthService.ReconcileSessionAsync` (on `ProfilesChanged`) also acquire the gate.
  - **Token path.** `GetTokenAsync` and the `ServiceClient` token provider do **not** take the gate, as `IAuthService` remarks and the class comment state. `GetTokenAsync` now goes through `AcquireSilentAsync`, a test seam that calls `pca.AcquireTokenSilent(...).ExecuteAsync(ct)` when not overridden, which is identical behaviour.
  - **ServiceClient cache.** `GetOrganizationServiceAsync` used to return `_cached` whenever `IsReady` was true. It now returns it only if `_auth.ActiveProfile.Id` and `EnvironmentUrl` match the profile id and URL recorded when the client was built. On a mismatch, or when the cached client is not ready, it disposes the old client and builds a new one.
  - **Unchanged.** The construction settings did not change: `useUniqueInstance: true`, `EnableAffinityCookie = false`, no `MaxRetryCount` or `RetryPauseTime`.
  - **Retry.** `RetrySelected` now re-checks under the lease that the active profile id equals the run's (`EnsureRetryTarget`). It throws `InvalidOperationException("The signed-in connection changed, ...")` otherwise. `CanRetrySelected` also requires `!IsRunning`.
- **Throughput effect:** `neutral` (inference).
  - The gate is acquired once per run. Nothing on the per-batch path takes it.
  - `GenerateAsync` calls `GetOrganizationServiceAsync` once per run (`WpfGenerationService.cs:45`). The extra identity check is an in-memory comparison.

    - The client is still reused across runs when the profile is unchanged.
    - Risk only: a long run makes a queued profile save, delete or sign-out wait, and a reconcile queued during the run fires as soon as it ends.
- **Notes:**
    - `sb-investigate.md` describes the gate correctly. It omits that profile save and delete also take the gate, and it omits the new identity-checked client cache.
    - Tests: `RunSessionGateTests` (3), `CachedClient_IsNotReusedForAnotherActiveProfile`, `Run_WaitsForInFlightSwitchAndReset`, `Switch_WaitsUntilGenerationFinishes`, `InFlightSignOut_BlocksRunStartupUntilResetCompletes`, `Retry_IsBlocked_AfterTheSessionMovesToAnotherProfile`.

`ee85321` · 2026-10-04 10:27 -0400 · Preflight required lookups before any write and at Review; keep written IDs on failure (CR-003)

- **Area:** pre-run reads (CPU-only checks, no I/O), lookups, error recording
- **Files:** `BulkCreator.cs`, `Core/Rules/RequiredLookupPreflight.cs` (new), `GenerationResult.cs`, `GenerateViewModel.cs`, `RunViewModel.cs`
- **Before → After:**
    - **Required-lookup check.** The T-25 check ("SystemRequired lookup with no constant, one-of or lookupRandom rule ⇒ throw") used to run inside `CreateEntityRecordsAsync` when each table was reached. Tables earlier in the order had already been written by then. It now runs as `PreflightTables(...)` over every table before `PrepareBogusRunOrThrowAsync`, so a missing rule fails the run before the first write. The `FieldAction.Fail` warnings move into the same up-front pass. The owner exemption changes from the `EdgeCaseValidator` `OwnerLookup` category to a literal `"ownerid"` name match.
    - **Failure handling.** `SubmitEntityBatchesAsync` now catches any non-cancel exception from `Parallel.ForEachAsync` into `failure`, collects the IDs already written, and `CreateAsync` rethrows it after the table. The outer catch went from `catch (OperationCanceledException) when (ct.IsCancellationRequested && ...)` to `catch (Exception ex) when (allCreatedRecords.Values.Any(ids => ids.Count > 0))`. That catch returns a result with `FatalError = ex.Message` instead of throwing.
    - **UI.** Review runs the same `RequiredLookupPreflight.FindUnsupplied`, and the sheet shows "Stopped".
- **Throughput effect:** `neutral`. The preflight is a metadata-only loop with no Dataverse call. The failure path stops the run at the same point as before; only the reporting changes.
- **Notes:**
    - `sb-investigate.md` row 3 lumps this commit in with "more pre-run `RetrieveMultiple` paging". This commit adds **no** reads; the reads come with `72e80ab`.
    - Tests: `CreateAsync_RequiredLookupGapOnSecondTable_WritesNothing`, `CreateAsync_FailureAfterAWrite_ReturnsWrittenIds`, `CreateAsync_FailureAfterBatchWrite_ReturnsCurrentTableIds`.

`72e80ab` · **2026-10-04 10:48 -0400 · Let a required lookup proceed when this run creates its target first (CR-003 amendment)**

- **Area:** lookups, pre-run reads, generation hot path
- **Files:** `BulkCreator.cs` (`PreflightTables`), `RequiredLookupPreflight.cs`, `GenerateViewModel.cs`
- **Before → After:**
  - `PreflightTables` now runs after `_topologicalSort.Sort(graph)` and computes `createdEarlier`: the tables sorted before this one with a count above 0.
  - A SystemRequired lookup with no rule is accepted when **every** target is in `createdEarlier` and is not the table itself (`IsCreatedEarlier`). Before, such a run threw.
  - `FindSuppliedByCreatedEarlier` then adds `tableRules.TryAdd(column, new LookupRandomRule())` into `validatedRules`, which is `config.FieldRules`.
  - Consequences of the implicit rule:
    - `PreparedLookupRun.PrepareAsync` treats it like any `lookupRandom` column. For each distinct target, it calls `ReadTargetAsync`: one sequential `RetrieveMultiple` page after another, `PageSize = 500`, up to `MaximumCandidatesPerTarget / PageSize` = 2 pages, each wrapped in `ThrottlePolicy.ExecuteAsync`, all before the first write. Results are cached per target in `byTarget`.
    - The targets are in the run, so their metadata is present and no `RetrieveEntityRequest` is issued.
    - Each generated row gains one `ResolveRuleValue` → `EvaluateLookupRandom` call per implicit column, through `explicitLookupAttrs`.
  - At this commit, `PreparedLookupRun.Get` still returns the **existing** environment rows whenever any were read. A required child therefore links to pre-existing parents, not to the rows this run created. `23970cf` changes that.
- **Throughput effect:** `lowers` time to first write slightly, only for profiles that have such a lookup (inference). Each target adds up to 2 sequential `RetrieveMultiple` calls of 500 rows, each able to enter the SDK and `ThrottlePolicy` retry stack. Write throughput (rows per request, concurrency) is unchanged.
  - Those profiles **failed outright before this commit** unless the user had added an explicit rule, and an explicit `lookupRandom` rule issues the same reads. So this commit adds reads only to runs that could not run before.
- **Notes:**
  - `sb-investigate.md` says implicit rules apply to "every SystemRequired lookup whose targets are created earlier in the run". That is incomplete: **all** targets must be created earlier, and a self-reference never qualifies.
  - Tests: `CreateAsync_RequiredLookupToTableCreatedEarlier_NeedsNoRule`, `CreateAsync_RequiredLookupToTableCreatedEarly_KeepsAnExplicitRule`, and 7 `RequiredLookupPreflightTests`.

`ff8fdb1` · **2026-10-04 13:06 -0400 · Offer Retry for batches lost to network faults, not only throttling**

- **Area:** retry-of-rejected, error recording
- **Files:** `ThrottlePolicy.cs`, `BulkCreator.cs`, `DeferredLookupBackfill.cs`, `BatchError.cs`, `Wpf/Services/Generation/RejectionClassifier.cs`

- **Before → After:**
  - **New classifier.** `ThrottlePolicy.IsTransient(Exception?)` returns true for `TimeoutException or OperationCanceledException or HttpRequestException or IOException`, for `EndpointNotFoundException or ServerTooBusyException`, and for a `ProtocolException` whose message contains `"(429)"`. It returns false for everything else, including `FaultException<OrganizationServiceFault>` and `(502)`.
  - **Where it is used.** `BatchError` gains `bool IsTransient = false`. `BulkCreator` sets it from `ex.InnerException` for a whole-batch failure, and `DeferredLookupBackfill` sets it from `ex.InnerException ?? ex`. `RejectionClassifier.IsRetryable` now returns true when `error.IsTransient || FaultCode is -2147015902 or -2147015903 or -2147015898 || message contains "429"`.
  - **What did not change.** `ThrottlePolicy.ExecuteAsync`, the in-process retry, is untouched. `EndpointNotFoundException` and `ServerTooBusyException` are now flagged retryable for the **user** Retry button, but they are still not retried automatically: they hit the generic `catch (Exception ex)` and fail on the first attempt.
- **Throughput effect:** `neutral` for a run. The flag affects only post-run classification and what the Retry button offers.
- **Notes:**
  - A `CommunicationException` that wraps `HttpRequestException` or `IOException` (a socket reset) is still classified non-transient here. `sb-investigate.md` says `256faa9` (Oct 28) addresses that.
  - `sb-investigate.md` attributes `IsTransient` to CR-002 (`22a5920`/`f14b561`). It comes from this commit.
  - Tests: `IsTransient_SeparatesNetworkFaultsFromDataFaults`, `CreateAsync_UnreachableEndpoint_MarksTheBatchTransient`, `Classifier_TransientNetworkFailureIsRetryable`.

`22a5920` · 2026-10-04 17:04 -0400 · Scope alternate-key values to the run and locate lost rows by absolute index (CR-002)

- **Area:** generation hot path (alternate keys), retry-of-rejected, error recording, settings+limits
- **Files:** `BulkCreator.cs`, `DeferredLookupBackfill.cs`, `BatchError.cs`, `GenerationConfig.cs`, `GenerationLimits.cs`
- **Before → After:**
  - **Alternate-key values.**
    - Before: `GenerateUniqueKeyValue` returned `"{entity}-{i:D8}"`, truncated from the left end (`[..max]`), for string keys and `recordIndex` for integer keys. The values were identical in every run.
    - After: `keyScope = SHA256.HashData(UTF8(config.AlternateKeyScope))` is computed once per table. String keys become `"{entity}-{hex(scope[0..4])}-{i:D8}"`, truncated keeping the right end (`key[^max..]`). Integer keys become `(int)(ReadUInt32BigEndian(scope) % 21_000) * 100_000 + i`, at most 2,099,999,999.
    - A new preflight throws when a string key's `MaxLength` is below 17.
  - **Retry row filter.** `GenerationConfig` gains `RowIndexes` and `AlternateKeyScope`. `PlannedRows` honours `RowIndexes`. `CreateEntityRecordsAsync` still generates all `recordCount` rows, then submits only `rowIndexes.Chunk(config.BatchSize)`. Batches are still `BatchSize` long.
  - **Error records.** `BatchError` drops `BatchIndex` and gains `RowIndexes` (the batch's rows, or `[rows[item.RequestIndex]]` per `ExecuteMultiple` fault).

    - **Validation.** `GenerationLimits.Validate` rejects retry rows outside `0..count-1`.
- **Throughput effect:** `neutral` at this commit. `AlternateKeyScope` defaults to `""` until `f14b561` sets it, so keys are still the same in every run. The per-row cost adds one short string format; the hash is computed once per table.
    - **Latent risk (inference):** integer alternate keys now reach about 2.1e9. If an integer key column has a `MaxValue` below that, every `CreateMultiple` in the table faults. The fallback on any `FaultException` (`BulkCreator.cs:482`) then replays each batch as `ExecuteMultiple`, and each row is rejected. That costs two requests per batch with zero rows written. The code comment flags it: "range-restricted integer keys are not honoured".
- **Notes:**
    - Tests: `AlternateKeys_AreStableWithinARun_AndDistinctAcrossRuns`, `AlternateKeyTooShort_FailsBeforeAnyCreateCall`, `RowFilter_WritesOnlyTheRejectedRows_WithTheirOriginalValues`.

`f14b561` · 2026-10-04 17:10 -0400 · Retry regenerates exactly the rejected rows in the run's key scope (CR-002)

- **Area:** retry-of-rejected, generation hot path (alternate keys)
- **Files:** `GenerateViewModel.cs` (`BuildConfig`), `RunViewModel.cs`, `RejectionGroup.cs`
- **Before → After:**
    - **Key scope.** `BuildConfig` sets `AlternateKeyScope = Guid.NewGuid().ToString("N")`, a fresh scope on every Start. From here on, alternate keys differ between runs.
    - **Retry config.**
        - Before: `RecordCounts = counts`, the number of rejected rows per table. That generated *new* rows 0..N-1 with fresh keys equal to the first run's rows 0..N-1.
        - After: `RecordCounts` keeps the **original** counts and `RowIndexes` holds the rejected rows. Only the retried tables are in `EntityLogicalNames`; `23970cf` changes that.
    - **Retryable groups.** `BuildGroup` marks a group retryable only when it is transient **and** has `RowIndexes`. Link-phase errors stop being retryable.
- **Throughput effect:**
    - `raises`, for repeated runs on tables that have alternate keys (inference). Before, run 2 produced the same keys as run 1, so every `CreateMultiple` faulted on a duplicate key. `SubmitBatchAsync` then fell back to `ExecuteMultiple`, where every row failed individually: two requests per batch and zero rows written. With a per-run scope, those runs write normally.
    - First runs are unaffected.
    - A retry now generates the entire table in memory to write a few rows. That is CPU time, bounded by `MaxRecordCount` (code comment: "ponytail: a retry regenerates the whole table").
- **Notes:**
    - Tests: `EachStart_GetsItsOwnAlternateKeyScope`, `RetrySelected_RegeneratesOnlyTheRejectedRows`, `RejectionWithoutRows_IsNotRetryable`.

`23970cf` · 2026-10-05 14:23 -0400 · Keep the first run's topology on retry, prefer in-run parents for implicit lookups, clear stale load warnings, and unskip gate tests (review WR-001..004)

- **Area:** lookups, retry-of-rejected, DOP (retry path only), pre-run reads
- **Files:** `BulkCreator.cs`, `PreparedLookupRun.cs`, `GenerationConfig.cs` (doc only), `RunViewModel.cs`, `JsonConnectionProfileService.cs`, `ConnectionManagerViewModel.cs`
- **Before → After:**
  - **In-run parents preferred.** `PreflightTables` now returns the `"table.column"` keys of the implicit rules it added. These are passed as `preferInRun` to `PreparedLookupRun.PrepareAsync`, which sets `PreparedLookupColumn.PreferInRun`.
    - `Get` now skips `Existing` when `PreferInRun` is set and builds the list from the in-run pool, sorted once and cached in `_resolved`. It falls back to `Existing` only when the pool is empty.
    - `PrepareAsync` **still reads** `ReadTargetAsync` for these columns before the first write. The reads become fallback-only but are not skipped.
  - **Skipped tables.** `PrepareAsync` now skips tables where `config.PlannedRows(table) <= 0`, and `inRunTargets` uses `PlannedRows`.
  - **Retry keeps the topology.** It now calls `RowIndexes.TryAdd(table, [])` for every first-run table and uses `_lastConfig with { RowIndexes = rows }`, keeping the original tables, counts and rules. `BulkCreator` skips writing tables with `config.PlannedRows(entityName) == 0`.
- **Throughput effect:** `neutral` for a first run. The read count is the same as at `72e80ab`; only which IDs get linked changes. For a **retry**, effects are `unclear` (inference):
  - **DOP hint.** The DOP hint is adopted only `if (entityIndex == 0 && config.MaxParallelism is null ...)` at the end of the loop body (`BulkCreator.cs:161`). A retry that skips the table at sort index 0 hits `continue` first, so the hint is never adopted and the retry stays at `effectiveDop = 8`.
  - **Warmup.** The `>= 5000` warmup test sums the original `RecordCounts`, so a small retry of a big run now does the `WhoAmI` warmup.
  - **Lookup reads.** `PreparedLookupRun` reads only for tables that have retried rows. Lookups whose parents were not retried draw from environment rows.
- **Notes:**
  - `sb-investigate.md` F4 correctly says the reads happen "even when the column is marked `PreferInRun`". Its line that `PrepareAsync` "issues a `RetrieveEntityRequest` for any target not already in the metadata set" never applies to implicit rules, whose targets are always in the run's metadata.
  - The commit also unskips two gate UI tests (`SEEDBOMB_SKIP_GATE_UI`) and clears a stale connection-store warning.
  - Tests: `CreateAsync_ImplicitRequiredLookup_PrefersThisRunsParents`, `CreateAsync_ChildOnlyRetry_KeepsTopologyAndRegeneratesTheRow`.

Out of scope in this window

Hash	Date	Subject	Why it does not touch the write path
`73a1595`	10-04 10:09	Show a terminal Failed sheet with the bound failure reason (WR-006)	`RunViewModel` end-of-run and failure display state only (`ApplyFailure`, indeterminate ring reset).
`9426a31`	10-04 16:53	Make alternate-key eligibility one entity-aware rule applied by every rule gate (WR-004)	Moves the "no rule on an alternate-key column" check from `EdgeCaseValidator` to `RuleEligibility` using `entity.Keys`. It runs once per configured rule in `ValidateConfiguredRules`; no per-row cost or I/O.
`2d713fc`	10-04 16:57	Rebuild cached MSAL clients when tenant, cloud or client ID changes (WR-013)	MSAL client fingerprint at sign-in. `GetTokenAsync` does not compute it.
`f1432e1`	10-04 16:57	Commit rule edits only after the profile write succeeds; add Retry save (WR-002)	Rules editor and profile save UI.
`e51015d`	10-04 16:57	Keep store warnings until shown; degrade on locked data files and failed migration (WR-007, WR-008)	Store load warnings. A locked `settings.json` now yields `AppSettings.Default` (batch 500, DOP auto), the same values as a missing file.
`b714687`	10-04 16:57	Name lookupRandom rules in the Profiles detail pane (IN-004)	One display string.
`55251d6`	10-04 17:06	Share one sign-in validator between the Connections editor and auth (WR-012, IN-005)	Touches `DataverseConnectionService`, but only replaces `new[] { $"{url}/.default" }` with `profile.DataverseScopes`, which is the same string. The sign-in validator now also requires `https`.
`c476892`	10-04 17:13	Read the live session from the auth service; clear selection on Cancel/New (WR-003, WR-011)	`SignedOut` handling moved into `MainWindowViewModel`; UI only.
`6aee1f8`	10-05 02:43	Surface draft store failures once per session; append crashes to the log (WR-009, IN-002)	Draft autosave snackbar. `CrashLog` appends instead of overwriting; it runs only on unhandled exceptions.
`2100472`	10-05 02:50	Inject the Generate board into Profiles; queue Open in board during a run (WR-001, WR-010, PR-004)	Profiles-to-board handoff; deferred while running.
`1a767f2`	10-05 02:54	Share the rule sampler between Rules and Review; hold one review snapshot (WR-005)	Review preview of 5 sample rows; not the run.
`6dbd96f`	10-05 02:58	Prune unused bulk, profile and board contracts; sort on typed row counts	The vague subject covers deleting the `IBulkCreator` interface (`GenerationPipeline` already used the concrete class), removing a legacy `AuthType` enum fix-up on connection load, and a Profiles sort key. No runtime change in Bulk.
`5cdb498`	10-05 07:55	Word the run snackbar from the sheet headline … (smoke item 2)	Snackbar text.
`037893b`	10-05 08:08	Label History runs with rejected rows Rejected … (smoke item 3)	History label.

Hash	Date	Subject	Why it does not touch the write path
`3cf3d40`	10-05 08:18	Give the rejected-row total one owner on GenerationResult	`RejectedRows => Errors.Sum(e => e.RowCount)` computed at end of run.
`7ff5870`	10-05 08:27	Report tables missing from the org in the import summary instead of the raw fault (smoke item 7)	Profile-import metadata fetch in the UI (`ObjectDoesNotExist` 0x80040217 tolerated); not part of a run.
`0daa012`	10-05 08:36	Disable Open in board when none of the profile's tables exist in this org	`CanExecute` predicate.
`420fe5d`	10-05 08:47	Base explicit InfoBar styles on the WPF-UI style ... (smoke item 8)	XAML style and InfoBar reopen.
`2847b7c`	10-05 08:59	Disable Load Profile until a profile is selected (smoke item 8a)	`CanExecute` predicate.
`5b7e6c3`	10-05 09:08	Disable Delete connection until the editor holds a saved profile (smoke item 9)	`CanExecute` predicate.
`b5c9aa5`	10-05 09:17	Point the FileLogger test at a temp folder ... (smoke item 10)	`FileLoggerProvider` gains a directory seam. `CurrentFile()` now calls `Directory.CreateDirectory(_directory)` per line, but `AppPaths.Logs` already did `Directory.CreateDirectory` per call before this, so per-line cost is unchanged. One open/close per log line under a global lock is pre-existing.
`c6ae090`	10-05 09:27	Replace swallowed cancels and an infinite delay in two gate tests ... (SW003, SW004)	Tests only.
`ab2a40d`	10-05 15:06	Remove WPF docs and strip manifest & locks	Diff is `.gitignore` only. The message mentions SKILL.md, manifest.json and skills-lock.json, which are not in the diff.
`3cbf429`	10-06 12:41	Add user and developer documentation ...	Markdown docs only (after v1.0.0).
`cf14ae3`	10-07 10:29	Map the WPF project to AnyCPU for x64 solution builds ...	`SeedBomb.slnx` platform mapping for IDE solution builds. The publish workflow still passes `-r win-x64` / `-r win-x86`.
`0c01714`	10-07 11:56	Require owner review via CODEOWNERS; add weekly Dependabot updates	`.github` config.
`2627251`	10-07 15:57 +0000	Bump actions/checkout from 5 to 7	CI workflow.
`968f6a6`	10-07 15:57 +0000	Bump actions/setup-dotnet from 4 to 6	CI workflow.

Parameter snapshot at 00e6c41

All paths are relative to the repo; line numbers come from `git show 00e6c41:<path>`.

- **Batch size.**
  - Default 500 (`src/SeedBomb.Core/Contracts/GenerationLimits.cs:15`, used by `GenerationConfig.cs:35`, `AppSettings.cs:17`) and max 1000 (`GenerationLimits.cs:12`).
  - Enforced by `GenerationLimits.Validate` (`GenerationLimits.cs:33-35`), called at `BulkCreator.cs:86`, and clamped in `GenerateViewModel.cs:866` and `AppSettings.cs:30`.
  - Batches come from `rowIndexes.Chunk(config.BatchSize)` (`BulkCreator.cs:356`). There is no adaptive sizing.
- **DOP.**
  - Settings default `DefaultDop = 0`, meaning auto (`AppSettings.cs:18`). The max is 16 (`GenerationLimits.cs:24`, clamped at `AppSettings.cs:31` and `GenerateViewModel.cs:867`).
  - `BuildConfig` maps 0 to `null` (`GenerateViewModel.cs:867`), so `effectiveDop = config.MaxParallelism ?? 8` (`BulkCreator.cs:113`).
  - With auto DOP, after the table at sort index 0 finishes, `effectiveDop = sc.RecommendedDegreesOfParallelism` if `> 0` (`BulkCreator.cs:161-172`). The hint is not clamped to 16, and it is never adopted if index 0 is skipped.
  - `WarmupAndAdoptRecommendedDopAsync` (`BulkCreator.cs:115-116`, `690-704`) runs `WhoAmI` when total rows `>= 5000` and DOP is auto. It only **logs** the hint and does not set `effectiveDop`, despite its name.
  - Concurrency is `Parallel.ForEachAsync` with `MaxDegreeOfParallelism = effectiveDop`, per table, across that table's batches (`BulkCreator.cs:378-433`). Tables run one after another in topological order.
- **App retry count and backoff.**
  - `GenerationConfig.MaxRetries = 3` (`GenerationConfig.cs:45`). The UI never sets it. That gives up to 4 attempts (`for attempt 0..maxRetries`, `ThrottlePolicy.cs:48`).
  - Delay is `min(60 s, 2 s * 2^attempt) + uniform(-2000, +2000) ms` jitter (`ThrottlePolicy.cs:17`, `128-134`): about 2 s, 4 s and 8 s, each ±2 s. `Retry-After` is not read.
- **Exceptions retried by `ThrottlePolicy`.**
  - Retried while `attempt < maxRetries`:
    - `TimeoutException`;
    - `OperationCanceledException` when the caller's token is **not** cancelled (an HttpClient timeout; IN-004, pre-window);
    - `HttpRequestException`;
    - `IOException`;
    - `ProtocolException` whose message contains `"(429)"` (`ThrottlePolicy.cs:61-86`).
  - Never retried, wrapped as `DataGenerationException` on the first occurrence:
    - `FaultException<OrganizationServiceFault>`, including service-protection codes (`:87-94`);

        - anything else (`:95-100`), for example `CommunicationException`, `EndpointNotFoundException`, `ServerTooBusyException`, or a `ProtocolException` with 502 or 503.
    - A caller cancel is rethrown (`:55-58`).
- `ServiceClient` **construction** (`src/SeedBomb.Wpf/Services/Dataverse/DataverseConnectionService.cs:72-80`).
    - Arguments: `new ServiceClient(instanceUrl, tokenProviderFunction: CreateTokenProvider(scopes), useUniqueInstance: true) { EnableAffinityCookie = false }`.
    - Not set: `MaxRetryCount`, `RetryPauseTime`, `MaxConnectionTimeout` or any timeout, and no logger. A repo-wide grep of `src` at `00e6c41` finds none of them, so SDK defaults apply.
    - `sb-investigate.md` gives those defaults as 10 retries, 5 s pause and a 4 min timeout for 1.2.x. Section 2.2 confirms them against the SDK's public source.
    - The token provider calls `_auth.GetTokenAsync(scopes, CancellationToken.None)` on every SDK request (`:129-130`); MSAL silent acquisition is cached.
    - The client is built only when no ready client is cached for the active profile id and URL (`:110-120`). A not-ready client is disposed and the build throws (`:92-97`).
    - None of these settings changed in this window.
- `CreateMultiple` **to** `ExecuteMultiple` **fallback** (`BulkCreator.cs:457-494`, unchanged in this window).
    - `CreateMultiple` is attempted while `_messageChecker.ShouldAttemptCreateMultiple(entity)` holds, an in-memory check with no request.
    - On a `DataGenerationException` whose inner fault code is `0x80040800`, it marks the entity unsupported and falls back. Later batches for that table go straight to `ExecuteMultiple`.
    - On **any other** inner `FaultException<OrganizationServiceFault>`, it logs and falls back for **this batch only**. That happens after `ThrottlePolicy` has failed the `CreateMultiple` without retrying it, which takes one attempt because faults are terminal.
    - The fallback is `ExecuteMultipleRequest` with one `CreateRequest` per row, `ContinueOnError = true`, `ReturnResponses = true` (`:521-570`), wrapped in its own `ThrottlePolicy.ExecuteAsync` with the same `MaxRetries`.
    - Non-fault failures (timeouts, network) after the retries do **not** fall back. They become one `BatchError` covering the whole batch (`:403-414`).
- **Dataverse reads before the first write** (in order):
    1. `GetOrganizationServiceAsync` (`WpfGenerationService.cs:45`) builds a `ServiceClient` only if none is cached for the active profile.
    2. `_metadata.GetEntitiesAsync(config.EntityLogicalNames)` (`GenerationPipeline.cs:47`), served by `DataverseMetadataService`'s `MemoryCache`, which is unchanged in this window.
    3. `PreparedLookupRun.PrepareAsync` (`BulkCreator.cs:105-106`; `PreparedLookupRun.cs:58-125`). For each distinct target of every explicit or implicit `lookupRandom` column on tables with planned rows:
        - an optional `RetrieveEntityRequest(EntityFilters.Entity)`, only when the target's metadata lacks `PrimaryIdAttribute`;
        - then up to 2 sequential `RetrieveMultiple` pages of 500 ordered by primary id (`PageSize = 500` at `:24`, cap 1000 per target).
        - Every call goes through `ThrottlePolicy`, and the reads are sequential.
    4. `PopulateCurrencyPoolAsync`: `RetrieveMultiple` of `transactioncurrency`, `TopCount = 10` (`BulkCreator.cs:617-642`).

5. `PopulateSystemUserPoolAsync`: `RetrieveMultiple` of `systemuser`, `isdisabled = false`, `TopCount = 25` (`BulkCreator.cs:648-674`).
6. If total rows `>= 5000` and DOP is auto: `WhoAmIRequest` (`BulkCreator.cs:690-704`).
   - Calls 4 to 6 are not wrapped in `ThrottlePolicy` and swallow non-cancel errors.
   - The run sheet shows the phase "Generating records" for all of steps 3 to 6 (`GenerationPipeline.cs:68`).
   - There is no `sdkmessagefilter` probe before writes. `IsUpdateMultipleAvailableAsync` runs only in the backfill phase, once per deferred-edge source entity (`DeferredLookupBackfill.cs:136`).

- **Gating.** `RunSessionGate` is acquired at `RunViewModel.cs:290`, before `GenerateAsync` (`:322`), and released at `:357`. The token path never takes it.
- **Progress.** `WpfGenerationService` forwards at most one non-terminal progress update per 100 ms (`ProgressGate`, `WpfGenerationService.cs:15`, `56-59`). `BulkCreator` reports once per batch (`BulkCreator.cs:423-432`). The per-batch success log is `LogDebug`, below the file logger's `Information` minimum, so it is not written. Warnings (fallback, retries, `ExecuteMultiple` failures) are written, one file open and close per line.

4.2 Window B: Oct 8 to Nov 1 2026 (00e6c41..b0654bc; ships as v1.0.1, v1.0.2, master, plus three branch-only commits)

The window holds 28 commits: 18 non-merge commits and 10 merges. All 10 merges are clean (`git show --remerge-diff` shows no conflict resolution), so every behaviour change is in a non-merge commit. No commit dated Oct 8 to Oct 27 exists on any ref apart from two unmerged Dependabot commits (listed below). The work starts on Oct 28. `v1.0.0` (56ec5ee) has the same `src/` and `Directory.Packages.props` as `00e6c41`, so "before" in this section is also "as in v1.0.0". `v1.0.1` and `v1.0.2` have the same runtime code. The only difference is `b008008`, which dropped the x86 publish. `PUBLISH.md` (29ba9f1) says 1.0.1 was withdrawn and replaced by 1.0.2, so a user on a "1.0.x" release most likely ran 1.0.2. The changes in this window that matter most for throughput are the DOP changes (`7cb61b5` shipped, `50280b4` and `837d63b` branch only) and the widened retry classification (`256faa9`, `35f4d23`). The other changes are observability that is off by default, logging, and per-row changes that are neutral.

First ref that contains each commit (`git merge-base --is-ancestor`, checked in order v1.0.1, v1.0.2, origin/master):

Commit	Date	First in	Subject
`256faa9`	2026-10-28 12:49 -0400	v1.0.1	Fixed
`44c0f48`	2026-10-28 13:12 -0400	v1.0.1	Fixed
`695b42d`	2026-10-30 07:32 -0400	v1.0.1	Bump Caching.Memory to 10.0.14 and map Wpf to x64
`35f4d23`	2026-10-30 07:32 -0400	v1.0.1	Phase 1 reliability fixes: metadata cache, retries, alternate keys
`d4d42ec`	2026-10-30 07:46 -0400	v1.0.1	Phase 2 observability: batch timing, phase intervals, run snapshot
`7cb61b5`	2026-10-30 07:52 -0400	v1.0.1	Phase 3: use the server DOP hint from the first table
`72fd797`	2026-10-30 08:14 -0400	v1.0.1	Reconcile on modifiedon, not createdon
`9fda97c`	2026-10-30 08:14 -0400	v1.0.1	Merge branch 'feat/phase2-observability' into feat/phase3-dop

Commit	Date	First in	Subject
`7e95da1`	2026-10-30 08:51 -0400	v1.0.1	Anchor reconcile to a pre-run server watermark
`f71fa30`	2026-10-30 08:51 -0400	v1.0.1	Merge branch 'feat/phase2-observability' into feat/phase3-dop
`828af2e`	2026-10-31 14:00 -0400	v1.0.1	Rename test locals so CodeQL's 'account' heuristic stops flagging table-name logs
`7883a21`	2026-10-31 14:09 -0400	v1.0.1	Move CodeQL to advanced setup and exclude test projects
`286cab0`	2026-10-31 14:14 -0400	v1.0.1	Merge remote-tracking branch 'origin/master' into feat/phase2-observability
`d637029`	2026-10-31 14:15 -0400	v1.0.1	Merge pull request #7 (fix/phase1-reliability)
`ba84481`	2026-10-31 14:20 -0400	v1.0.1	Merge remote-tracking branch 'origin/master' into feat/phase3-dop
`d8f4d0a`	2026-10-31 14:21 -0400	v1.0.1	Merge pull request #8 (feat/phase2-observability)
`78a86d2`	2026-10-31 14:25 -0400	v1.0.1 (is the tag)	Merge pull request #9 (feat/phase3-dop)
`b008008`	2026-10-31 15:28 -0400	v1.0.2	Drop the x86 release build
`4feba6a`	2026-10-31 15:33 -0400	v1.0.2 (is the tag)	Merge pull request #10 (ci/drop-x86)
`29ba9f1`	2026-10-31 15:45 -0400	master (untagged)	Document immutable release tags in PUBLISH.md
`e1d0553`	2026-10-31 15:56 -0400	master (untagged)	Merge pull request #11 (docs/publish-immutable-tags)
`0a46583`	2026-11-01 02:45 -0500	master (untagged)	Document setting Created On through a Record Created On rule
`449cac2`	2026-11-01 02:46 -0500	master (untagged)	Stop backdating createdon: write overriddencreatedon only when a rule sets it
`339cb08`	2026-11-01 02:47 -0500	master (untagged)	Point the Created On picker row to Record Created On rules
`b4433d2`	2026-11-01 03:22 -0500	master (untagged, is origin/master)	Merge pull request #12 (fix/createdon-rules)
`50280b4`	2026-11-01 04:08 -0500	branch only	Map an emptied parallelism box to 0 so it means automatic
`837d63b`	2026-11-01 04:10 -0500	branch only	Cap an explicit DOP at the server hint and report it on the result
`b0654bc`	2026-11-01 04:11 -0500	branch only	Show run notices in the summary banner and activity log

The three branch-only commits are on `fix/dop-cap-at-server-hint` and `ccr-7fdbebbc-r7v8km` (both at `a5fbbe5`) and are ancestors of HEAD.

Commits Oct 8 to Oct 27 on any ref (`git log --all --since=2026-10-08 --until=2026-10-28`): only the Dependabot commits `b440811` (Oct 14) and `6b266ed` (Oct 28 00:16 UTC, which is Oct 27 20:16 EDT). Neither is merged.

Unmerged Dependabot branches (none is in a shipped build, and none touches `Microsoft.PowerPlatform.Dataverse.Client`):

Ref	Commit	Date	Base	Bump
`origin/dependabot/nuget/nuget-f92cf0a58d`	`ff6207c`	2026-10-07	`0c01714`	`System.Security.Cryptography.Xml` 10.0.10 → 10.0.12
`origin/dependabot/nuget/nuget-a55301f27d`	`b440811`	2026-10-14	`00e6c41`	`System.Security.Cryptography.Xml` 10.0.10 → 10.0.13
`origin/dependabot/nuget/nuget-9a18e4882f`	`6b266ed`	2026-10-28 (UTC)	`00e6c41`	`System.Security.Cryptography.Xml` 10.0.10 → 10.0.14 (the same pin `256faa9` then made by hand)
`origin/dependabot/nuget/nuget-77c1b23c9f`	`a68ceе6`	2026-11-11 (after the window)	`b4433d2`	8 updates: `Microsoft.Extensions.Caching.Memory` 10.0.14 → 11.0.0, `Microsoft.Extensions.Hosting` 10.0.12 → 11.0.0, `Microsoft.Identity.Client` and `.Extensions.Msal` 4.90.0 → 4.92.0, `System.Security.Cryptography.Xml` 10.0.14 → 11.0.0, `Microsoft.NET.Test.Sdk` 18.10.1 → 18.11.1, `Moq` 4.20.72 → 4.21.0, `xunit.v3` 4.0.0 → 4.0.3

`Microsoft.PowerPlatform.Dataverse.Client` is 1.2.27 at both ends of the window.

`256faa9` · 2026-10-28 12:49 -0400 · Fixed

- **First shipped in:** v1.0.1
- **Area:** retry, packages
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `Directory.Packages.props`, `src/SeedBomb.{Core,Bulk}/*.csproj` (plus three test csproj files), `.gitignore`, deleted `src/SeedBomb.Wpf/Properties/PublishProfiles/FolderProfile.pubxml`
- **Before → After:**
  - Retry in `ThrottlePolicy.ExecuteAsync`: a new `catch (CommunicationException ex) when (attempt < maxRetries && IsWrappedNetworkFault(ex))`, where `IsWrappedNetworkFault` is `ex.InnerException is HttpRequestException or IOException`. Before, a `CommunicationException` (including a non-429 `ProtocolException`) that wraps a socket reset fell into the final `catch (Exception)`, was wrapped in `DataGenerationException`, and failed the batch on the first attempt. That batch had no `ExecuteMultiple` fallback, because the fallback only catches an inner `FaultException<OrganizationServiceFault>`. After, it is retried up to `maxRetries` (3) times with the existing `ComputeDelay` backoff, and each retry is logged at Warning as "Network fault for {Entity} on attempt n/N".
  - Classification in `IsTransient`: `ProtocolException p => IsRateLimited(p) || IsWrappedNetworkFault(p)` and a new `CommunicationException c => IsWrappedNetworkFault(c)`. These rows are now reported as transient, so the user is offered Retry. A bare `CommunicationException` such as "Unrecognized message version" is still not transient.
  - Packages: the central `System.Security.Cryptography.Xml` pin goes from 10.0.10 to 10.0.14 (`GlobalPackageReference`, `net10.0` only). Core, Bulk and the test projects add `PackageVersion Update` / `PackageReference Update` with `PrivateAssets=all`. This is not on the HTTP path.

    - Repository housekeeping: `.gitignore` now ignores `*.md` (so a new note such as this file has to be added with `git add -f`) and the local `FolderProfile.pubxml`. The tracked `FolderProfile.pubxml` (self-contained, single-file, `PublishReadyToRun=false`) is deleted. The release workflow uses its own `GitHubRelease-x64` profile, so the shipped exe is not affected.
- **Throughput effect:** `neutral` on a healthy connection, because nothing new runs unless a wrapped socket fault occurs. When resets do occur, more rows end up written, but each affected batch holds a DOP slot through up to three sleeps (about 2 s, 4 s and 8 s, each ±2 s) plus the repeated call. A reset in the middle of an upload can also come after the server has committed the `CreateMultiple`, and the resend can then duplicate rows or hit alternate-key duplicates. That last point is inference: the code cannot know whether the request committed.
- **Notes:** Tests: `ExecuteAsync_ConnectionResetWrappedInCommunicationException_IsRetried`, plus two `TransientCases` rows. The subject "Fixed" says nothing. The body is accurate. Phase 1 (`35f4d23`) folds this catch into a single `RetryReason` switch.

`695b42d` · 2026-10-30 07:32 -0400 · Bump Caching.Memory to 10.0.14 and map Wpf to x64

- **First shipped in:** v1.0.1
- **Area:** packages
- **Files:** `Directory.Packages.props`, `SeedBomb.slnx`
- **Before → After:** `Microsoft.Extensions.Caching.Memory` 10.0.12 → 10.0.14 (a patch release). This is the `MemoryCache` behind `DataverseMetadataService`. `SeedBomb.slnx` now gives the Wpf project the `*|x64` platform mapping, which pairs with `44c0f48` adding `x64` to the Wpf `<Platforms>`. The `System.Security.Cryptography.Xml` line is only reformatted.
- **Throughput effect:** `neutral`. The cache only holds entity metadata, which is read once per run and normally hit. The platform mapping changes the solution build. Release publishing already used `win-x64`.
- **Notes:** None. `sb-investigate.md` does not mention this bump.

`35f4d23` · 2026-10-30 07:32 -0400 · Phase 1 reliability fixes: metadata cache, retries, alternate keys

- **First shipped in:** v1.0.1
- **Area:** retry, progress+logging, pre-run reads, generation hot path, gating (Review step)
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Core/Rules/ScopedAlternateKey.cs` (new), `src/SeedBomb.Core/Metadata/DataverseMetadataProvider.cs`, `src/SeedBomb.Wpf/Services/Dataverse/DataverseMetadataService.cs`, `src/SeedBomb.Wpf/Services/Generation/WpfGenerationService.cs`, `src/SeedBomb.Wpf/ViewModels/GenerateViewModel.cs`
- **Before → After:**
    - **Retry set widened.** Five `catch` blocks (timeout/OCE, `HttpRequestException`, `IOException`, `ProtocolException` 429, wrapped `CommunicationException`) become a single `catch (Exception ex) when (attempt < maxRetries && RetryReason(ex) is { } reason)`. `RetryReason` returns "Timeout" for `TimeoutException` or `OperationCanceledException` (a caller cancel is still rethrown first), "Network fault" for `HttpRequestException` or `IOException`, **"Endpoint not found" for** `EndpointNotFoundException`, **"Server too busy" for** `ServerTooBusyException`,

"HTTP 429" for a `ProtocolException` whose message contains `(429)`, and "Network fault" for a `CommunicationException` that wraps `HttpRequestException` or `IOException`. Before, `EndpointNotFoundException` and `ServerTooBusyException` were reported transient by `IsTransient` but were **not** retried by `ExecuteAsync` (the final `catch (Exception)` failed them at once). They now get up to 3 retries with backoff. `IsTransient` is now `RetryReason(ex) is not null`, so the two can no longer disagree.
  - **Retry logging.** Every retry now logs at Warning with the reason, "attempt N of M" (M is `maxRetries + 1`), the failed call's duration from a `Stopwatch` taken per attempt, the exception type and message, and the delay. Before, direct `HttpRequestException` and `IOException` retries logged nothing.
  - **Metadata cache on a connection reset.** `DataverseMetadataService.ResetCacheAsync` no longer disposes the old `MemoryCache`, which is left for the GC. `Dispose` is idempotent through `_disposed`, and the `SemaphoreSlim` is no longer disposed. `DataverseMetadataProvider` wraps cache reads and writes in `TryGetCached` / `SetCached`, which treat `ObjectDisposedException` as a miss or a no-op. Before, a reset during a run's metadata read could throw `ObjectDisposedException`, which became a `SchemaException`. The number of metadata calls is unchanged (30-minute cache).
  - **Alternate keys, per row.** `BulkCreator.GenerateUniqueKeyValue` is replaced by `ScopedAlternateKey.Generate(attr, entityName, i, recordCount, keyScope)`. String keys are unchanged. Integer keys now honour the column's `MinValue`/`MaxValue`: blocks of `IntegerStride = 100_000`, up to `IntegerRunOffsets = 21_000` of them, or a sliding window when the range is narrower than two blocks. On the default int range this gives the same values as before (`scope % 21000 * 100000 + i`). The work per row is a few arithmetic operations.
  - **Preflight gating.** `PreflightTables` now calls `ScopedAlternateKey.FindUnsupported(entityName, meta, count)` and throws before any write when a string key is shorter than 17 characters (as before) **or** an integer key's range cannot hold `recordCount` distinct values (new). Key columns are now chosen by `meta.Keys` plus `FieldFilter.ShouldGenerateField` instead of the edge-case validator. The Review step (`GenerateViewModel`) shows the same messages as errors that block Start.
  - **Completion log.** `WpfGenerationService` logs "Generation cancelled", "failed", "finished with errors" or "complete" depending on the result, instead of always "Generation complete".
- **Throughput effect:** `unclear`. On the healthy path it is `neutral`: one `Stopwatch.GetTimestamp()` per attempt, and per-row key generation of the same cost. The one behaviour that can lower rows per second is the two newly retried types. In WCF, `ServerTooBusyException` is the exception mapped from HTTP 503, so on an overloaded server a batch that used to fail fast can now sleep up to about 14 s (2 + 4 + 8 s, each ±2 s) across 3 retries, on top of any retries the SDK already did inside each attempt. Whether `ServiceClient` 1.2.27 surfaces 503 as `ServerTooBusyException` on its path after its own retries is **not verified** here. It raises rows written, but lengthens stalls during incidents.
- **Notes:** Tests: `ExecuteAsync_FaultIsTransientReports_IsRetried` (theory over the transient cases), `ExecuteAsync_DirectHttpRequestException_LogsTheRetry`, three integer-key tests in `BulkCreatorTests`, two metadata-service dispose tests, and a Review alternate-key test. The commit message is accurate. It understates the retry change: "EndpointNotFoundException and ServerTooBusyException are now retried" is the only real widening, and the message presents it as a side effect of unifying the classification.

`d4d42ec` · **2026-10-30 07:46 -0400 · Phase 2 observability: batch timing, phase intervals, run snapshot**

- **First shipped in:** v1.0.1
- **Area:** progress+logging, client config, reconcile, settings+limits
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk/RunDiagnostics.cs` (new), `src/SeedBomb.Bulk/GenerationPipeline.cs`, `src/SeedBomb.Core/Contracts/GenerationConfig.cs`, `src/SeedBomb.Wpf/Services/Dataverse/DataverseConnectionService.cs`, `src/SeedBomb.Wpf/Services/Dataverse/MinimumLevelLogger.cs` (new), `src/SeedBomb.Wpf/Services/Generation/WpfGenerationService.cs`, `src/SeedBomb.Wpf/Services/Settings/AppSettings.cs`, `src/SeedBomb.Wpf/ViewModels/SettingsViewModel.cs`, `SettingsPage.xaml`
- **Before → After:**
  - **Per-batch timing line (always on).** `SubmitBatchAsync` now takes a `BatchTrace`, and each `CreateMultiple` or `ExecuteMultiple` attempt is wrapped in `trace.CallAsync(api, ...)`, which counts attempts, records the API path ("CreateMultiple", "ExecuteMultiple" or "CreateMultiple>ExecuteMultiple") and times the last call. After each batch, `LogBatchTiming` writes one Information line: `[timing] batch {Entity} {i}/{n}: {Rows} rows via {Api}, {Attempts} attempt(s), DOP {Dop}, last call {ms}, total {ms}, {ok|partial|failed}, fault {type|0x code|none}`. The line runs inside the `Parallel.ForEachAsync` body. The file sink (`FileLoggerProvider`, unchanged) does `File.AppendAllText` plus `Directory.CreateDirectory` per line under one global lock.
  - **Phase lines (always on).** `[timing] phase` lines for `metadata` and `graph` (in `GenerationPipeline`), `preflight`, `prepare` (Bogus prep, lookup prep and the currency/systemuser pools), `warmup`, `generate` and `submit` per table, `backfill`, and `associate`.
  - **Run snapshot (always on).** `RunDiagnostics.LogSnapshot` runs once per run, before the first write. It logs the app, `Dataverse.Client` and `Caching.Memory` versions (by enumerating loaded assemblies), batch size, configured DOP ("auto" when null), `sc.RecommendedDegreesOfParallelism`, `sc.EnableAffinityCookie`, and the first-attempt API per table from `ShouldAttemptCreateMultiple`, which is in-memory. It makes no Dataverse call.
  - **SDK logger (opt-in).** A new `AppSettings.RunDiagnostics` (default `false`) and a Settings toggle. When it is on, `DataverseConnectionService` passes `logger: new MinimumLevelLogger(..., LogLevel.Warning)` to the `ServiceClient` constructor. This applies only to the next client built, since the client is cached. With the setting off, `logger:` is `null`, which is the same as before. No other `ServiceClient` setting changes.
  - **Post-run reconcile (opt-in).** `WpfGenerationService` reloads settings for every run and sets `GenerationConfig.ReconcileAfterRun`. When it is true, `RunDiagnostics.ReconcileAsync` runs after the associate phase, and also from the catch path of a failed or cancelled run that wrote rows. It issues one aggregate FetchXML `count` per planned table, one after another, with `createdon ge (runStart - 1 min)` and `createdby eq-userid`, directly on `_service` (not through `ThrottlePolicy`), under its own 30 s `CancellationTokenSource` (`ReconcileTimeout`). Failures are logged, not thrown.
- **Throughput effect:** `neutral` with the default settings (inference). Per batch it adds one `BatchTrace` allocation, two timestamps and one synchronous file append. That is milliseconds against `CreateMultiple` calls that take seconds, but it is serialized through one global lock. With Run diagnostics on, the run gains up to N extra count queries after the last write (capped at 30 s in total). That lowers whole-run rows per second, not per-batch throughput. On the failed or cancelled path, `GenerationResult.Elapsed` is computed after the reconcile, so a rate derived from it includes the reconcile time. On the success path `Elapsed` is computed before it.

- **Notes:** Tests: `CreateAsync_LogsOneSnapshot_PhaseIntervals_AndOneTimingRecordPerBatch`, `CreateAsync_WithReconcile_LogsRequestedAcknowledgedAndObserved`, and a settings round-trip test. The `[timing] batch` line is what any throughput comparison should be measured against. It does not exist in v1.0.0, so a v1.0.0 log cannot be compared batch by batch. The reconcile filter is changed by `72fd797` and `7e95da1`.

`7cb61b5` · 2026-10-30 07:52 -0400 · Phase 3: use the server DOP hint from the first table

- **First shipped in:** v1.0.1
- **Area:** DOP, pre-run reads
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs` (tests in `BulkCreatorTests.cs`)
- **Before → After:**
  - **DOP resolution.** Before (v1.0.0 to `d4d42ec`): `var effectiveDop = config.MaxParallelism ?? 8;` was set once, before the table loop. Only after the table at **topological index 0** finished (`entityIndex == 0`, reached only if that table was not skipped by a `continue` and did not throw), and only when `config.MaxParallelism is null`, was `effectiveDop` replaced by `sc.RecommendedDegreesOfParallelism` if that value was `> 0`. So the first table always ran at 8. A single-table run ran entirely at 8, and so did any run whose index-0 table was skipped (count 0, `PlannedRows == 0` on a Retry, or missing metadata). After: `ResolveDop(config, entityName)` runs at the top of **every** table. It returns `config.MaxParallelism` if set. Otherwise it returns `RecommendedDopHint` (`sc.RecommendedDegreesOfParallelism`, now `protected virtual`) if that is `> 0`, **without clamping it to** `GenerationLimits.MaxDop` **(16)**: a value above 16 is logged at Information and used as given. Otherwise it returns `const int DefaultDop = 8`. The post-table-0 update is removed.
  - **Warmup.** Before: `if (config.RecordCounts.Values.Sum() >= 5000 && config.MaxParallelism is null)` sent `WhoAmIRequest`, and only logged the hint (the method name says "Adopt", but it adopted nothing). After: `if (config.MaxParallelism is null && config.PlannedTotal > 0)`, so every automatic-DOP run that writes at least one row sends one `WhoAmIRequest` before the first table. It goes directly through `_service.ExecuteAsync` (not `ThrottlePolicy`). Failures other than OCE are logged at Debug and ignored.
- **Throughput effect:** `unclear`, and it can **lower** throughput relative to v1.0.0. Concurrency for the first table, and for the cases above where v1.0.0 stayed at 8 for the whole run, now equals the server hint. If the environment's `x-ms-dop-hint` is below 8, those tables run fewer concurrent 500-row `CreateMultiple` calls than v1.0.0 did (for example 4 instead of 8, which halves in-flight rows). If the hint is above 8, concurrency rises, possibly above 16. Microsoft's guidance is that going above the hint worsens performance, so the net effect on rows per second depends on the environment, which is unknown here. Two further points, both inference: (a) per the public SDK source (`ConnectionService.RecommendedDegreesOfParallelism { get; set; } = 5`, and `ServiceClient.RecommendedDegreesOfParallelism => _connectionSvc.RecommendedDegreesOfParallelism`), a real `ServiceClient` reports **5**, not 0, until a hint header has been seen. The default of 5 is confirmed in the SDK's public master source (section 2.2; the 1.2.27 tag was not checked separately), so the `hint <= 0 → 8` fallback is dead code in production, and any environment that sends no hint runs at 5 instead of v1.0.0's 8. (b) Small runs (under 5,000 rows) gain one extra round trip.
- **Notes:** Tests: `Dop_ServerHint_IsUsedFromTheFirstTable`, `Dop_ExplicitConfig_OverridesTheHint`, `Dop_NoHint_FallsBackToEight` (uses a test override returning 0, which a real `ServiceClient` probably never returns), `Dop_HintAboveMaxDop_IsUsedUnclampedAndLogged`,

`Dop_HintThatChangesBetweenTables_IsReadPerTable`, `Warmup_RunsBeforeTheFirstTable_WheneverDopIsAutomatic`, `Warmup_IsSkipped_WhenDopIsConfigured`. The message matches the diff. The pre-run reads (lookup prep, currency and systemuser pools) already go through the same client before the warmup, so `ServiceClient` has probably recorded a hint even without the WhoAmI (inference).

`72fd797` · 2026-10-30 08:14 -0400 · Reconcile on modifiedon, not createdon

- **First shipped in:** v1.0.1
- **Area:** reconcile
- **Files:** `src/SeedBomb.Bulk/RunDiagnostics.cs`, `src/SeedBomb.Core/Contracts/GenerationConfig.cs` (doc comment and BOM only)
- **Before → After:** The opt-in post-run count FetchXML condition changes from `createdon ge {since}` to `modifiedon ge {since}`, keeping `createdby eq-userid`. The log text now reads "written since".
- **Throughput effect:** `neutral`. It runs only with Run diagnostics on, after the last write, and the query count is unchanged.
- **Notes:** The message says "generated rows set overriddencreatedon, so Dataverse stamps createdon with a past date". That was true of v1.0.0 to v1.0.2 and is fixed later by `449cac2`. This is also the commit whose message says "Found in the first live run". `sb-investigate.md` section 4 attributes that phrase to "the Phase 2 commit message".

`7e95da1` · 2026-10-30 08:51 -0400 · Anchor reconcile to a pre-run server watermark

- **First shipped in:** v1.0.1
- **Area:** reconcile, pre-run reads
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk/RunDiagnostics.cs`
- **Before → After:**
  - **New pre-run read (opt-in).** `RunDiagnostics.CaptureWatermarksAsync` is called in `CreateAsync` after `LogSnapshot` and before the first table. When `ReconcileAfterRun` is set, it issues one `RetrieveMultiple` per planned table: `QueryExpression(table) { ColumnSet("modifiedon"), TopCount = 1 }`, `createdby EqualUserId`, ordered by `modifiedon` descending. The reads run one after another, directly on `_service`, under the run's `ct`, with no timeout of their own and no `ThrottlePolicy`. A failure leaves the table out of the dictionary; a run cancel is rethrown.
  - **Post-run count.** The count changes from `modifiedon ge (runStart - 1 min)` to `modifiedon gt {watermark}`. When the watermark is null (the caller has no rows in the table yet) there is **no** time filter, so all of the caller's rows in the table are counted, and past 50,000 that fails with the aggregate limit and is logged as unavailable. A table with no watermark is reported "unavailable (no pre-run watermark)" without a query. `ReconcileClockMargin` is removed.
- **Throughput effect:** `neutral` with the default settings. With Run diagnostics on, it adds N sequential reads **before the first write**. On a large table, `ORDER BY modifiedon DESC` filtered by `createdby` may not be index-backed and could take seconds (inference). That time shows up as a longer gap before the first batch, not as lower rows per second during submission.

- **Notes:** Tests: `CreateAsync_WithReconcile_CountsOnlyRowsWrittenAfterThePreRunWatermark`, `CreateAsync_WithReconcile_ReportsATableWhoseWatermarkReadFailed`. Merged into the Phase 3 branch by `f71fa30`, cleanly.

`449cac2` · 2026-11-01 02:46 -0500 · Stop backdating createdon: write overriddencreatedon only when a rule sets it

- **First shipped in:** master (untagged)
- **Area:** generation hot path
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Core/Metadata/FieldFilter.cs` (tests in `RuledGenerationTests.cs`)
- **Before → After:** In `CreateEntityRecordsAsync`, both per-row loops (`attributesToGenerate` and `specialHandlingAttrs`) gain `else if (FieldFilter.IsRuleOnly(attr)) continue;` after the rule lookup. `IsRuleOnly` matches a `HashSet` holding only `"overriddencreatedon"`. Before: every generated row carried a random `overriddencreatedon`, so Dataverse backdated `createdon`. After: the generator is still called (to keep the Faker stream stable), but the value is dropped unless a profile rule targets the column.
- **Throughput effect:** `neutral`. Each row is one attribute smaller and there is one extra hash lookup per attribute per row, which is negligible either way. No change in requests.
- **Notes:** Tests: `Record_created_on_is_left_unset_without_a_rule`, `Record_created_on_rule_sets_the_date_and_leaves_other_columns_unchanged`. It changes the data written (`createdon` is now the real write time), which matters for anyone comparing `createdon`-based counts before and after.

`50280b4` · 2026-11-01 04:08 -0500 · Map an emptied parallelism box to 0 so it means automatic

- **First shipped in:** branch only
- **Area:** settings+limits, DOP
- **Files:** `src/SeedBomb.Wpf/Converters/NullToZeroConverter.cs` (new), `src/SeedBomb.Wpf/Resources/Shared.xaml`, `src/SeedBomb.Wpf/Views/Pages/GeneratePage.xaml`, `src/SeedBomb.Wpf/Views/Pages/SettingsPage.xaml`
- **Before → After:** The Generate page's `Parallelism` `NumberBox` (`Value="{Binding MaxParallelism}"`) and the Settings page's `Max parallelism` box (`DefaultDop`) now bind through `NullToZeroConverter`. `ConvertBack` maps `null`, or a non-finite number, to `0`, and other numbers to `(int)Math.Round(number)`. `GenerateViewModel.BuildConfig` (unchanged) maps `0` to `MaxParallelism = null`, which is automatic. Before, the box's `double?` value was bound straight to the `int` property. Clearing the box (which shows the "auto" placeholder) is most likely a failed binding conversion that leaves the previous number in the view model, so the run used that explicit DOP while the UI read "auto". This is inference from WPF binding semantics. The new STA test only checks the new path.
- **Throughput effect:** `unclear`. For a user who cleared the box, the run moves from an explicit DOP (whatever was last typed or saved, up to 16, with no WhoAmI warmup) to automatic: a WhoAmI, then the per-table hint from `7cb61b5`. That lowers concurrency when the old explicit value was above the hint and raises it when it was below.
- **Notes:** Tests: `ConverterTests` (converter round-trip), `GenerateAsync_ZeroParallelism_SendsNullSoTheRunIsAutomatic`, `ParallelismBoxBindingStaTests.ClearingTheBox_ResetsTheSourceToZero`. No commit message body.

`837d63b` · 2026-11-01 04:10 -0500 · Cap an explicit DOP at the server hint and report it on the result

- **First shipped in:** branch only
- **Area:** DOP, progress+logging
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Core/Contracts/GenerationConfig.cs` (doc), `src/SeedBomb.Core/Contracts/GenerationResult.cs`
- **Before → After:** `ResolveDop(config, entityName, notices)` reads `RecommendedDopHint` first. Explicit `MaxParallelism` is returned unchanged only if `hint <= 0 || configured <= hint`. Otherwise it logs a Warning "Parallelism {Configured} for {Entity} is above the server hint {Hint}; using the hint", adds a deduplicated notice "Parallelism capped at {hint} (the server's recommended maximum); {configured} was requested." to a run-level list, and **returns the hint**. The check runs per table, so a hint that changes between tables re-caps. New `GenerationResult.Notices` (an `IReadOnlyList<string>`, default empty) is filled on both the success and the catch paths. The warmup condition is unchanged (`MaxParallelism is null && PlannedTotal > 0`), so with an explicit DOP **no WhoAmI is sent**, and the cap uses whatever hint `ServiceClient` last recorded from the run's earlier calls (lookup prep, the currency and systemuser pools, metadata).
- **Throughput effect:** `lowers` concurrency for any explicit DOP above the hint (for example 16 → 4 on an environment hinting 4). Rows per second is `unclear`: Microsoft says calls above the hint do not help, but this is the one change in the window that can cut a user's explicitly requested parallelism. If the SDK reports its default of 5 when no header has been seen (see `7cb61b5`; confirmed in the SDK's public master source, section 2.2), any explicit DOP above 5 is capped at 5 even without a server hint.
- **Notes:** Tests: `Dop_ExplicitConfigBelowTheHint_IsUsed`, `Dop_ExplicitConfigAboveTheHint_IsCappedAtTheHintAndReported`, `Dop_ExplicitConfigEqualToTheHint_IsNotReported`, `Dop_ExplicitConfigWithNoHint_IsUsedAsConfigured`. `GenerationLimits.Validate` still accepts 1 to 16. The notice is shown in the UI by `b0654bc`. Combined with `50280b4` and `7cb61b5`, every branch run uses at most the hint, unless the hint is 0 or below (unlikely with a real client).

Out of scope in this window

Hash	Date	Subject	Reason
`44c0f48`	2026-10-28	Fixed	Post-run UI only: "Run cancelled" banner (`ApplyCancelled`), `RunRecord.Cancelled` and its History status, historical header text. Also adds `x64` to the Wpf `<Platforms>` and widens the Debug/Release property-group conditions to `AnyCPU` or `x64` (no optimisation flag changes: Release sets no `Optimize`, so the SDK default applies as before). No write-path change.
`828af2e`	2026-10-31	Rename test locals so CodeQL's 'account' heuristic stops flagging table-name logs	Test-only rename in `RuledGenerationTests.cs`.
`7883a21`	2026-10-31	Move CodeQL to advanced setup and exclude test projects	CI workflow only.

Hash	Date	Subject	Reason
`b008008`	2026-10-31	Drop the x86 release build	Release workflow, `PUBLISH.md`, and the deleted `GitHubRelease-x86.pubxml` only. The x64 build is unchanged.
`29ba9f1`	2026-10-31	Document immutable release tags in PUBLISH.md	Docs only.
`0a46583`	2026-11-01	Document setting Created On through a Record Created On rule	`field-rules.md` plus a comment-only edit in `RunDiagnostics.cs`.
`339cb08`	2026-11-01	Point the Created On picker row to Record Created On rules	`RuleEligibility.Classify` returns the new reason `CreatedOnViaOverride` for `createdon` before the `IsValidForCreate == false` check. `createdon` was already ineligible (`NotCreatable`), so only the reason text changes (in the picker, `RuleValidator` and `ProfileImport`). Nothing on the per-row path changes. Test fixtures now use `overriddencreatedon`.
`b0654bc`	2026-11-01	Show run notices in the summary banner and activity log	`RunViewModel` appends `result.Notices` to `OutcomeDetail` and the activity log once per run, after the run.
`9fda97c`	2026-10-30	Merge phase2 into phase3	Clean merge (empty remerge diff).
`f71fa30`	2026-10-30	Merge phase2 into phase3	Clean merge.
`286cab0`	2026-10-31	Merge origin/master into phase2	Clean merge.
`d637029`	2026-10-31	Merge PR #7 (phase1)	Clean merge.
`ba84481`	2026-10-31	Merge origin/master into phase3	Clean merge.
`d8f4d0a`	2026-10-31	Merge PR #8 (phase2)	Clean merge.
`78a86d2`	2026-10-31	Merge PR #9 (phase3), tag v1.0.1	Clean merge.
`4feba6a`	2026-10-31	Merge PR #10 (drop x86), tag v1.0.2	Clean merge.
`e1d0553`	2026-10-31	Merge PR #11 (docs)	Clean merge.
`b4433d2`	2026-11-01	Merge PR #12 (createdon rules), origin/master	Clean merge.

Parameter snapshot at b4433d2 (origin/master) and at b0654bc

Between `b4433d2` and `b0654bc`, the runtime code differs only in `BulkCreator.ResolveDop` (`837d63b`), `GenerationResult.Notices`, the `NullToZeroConverter` binding (`50280b4`), and the notices display. `ThrottlePolicy.cs`, `DataverseConnectionService.cs`, `PreparedLookupRun.cs`, `DeferredLookupBackfill.cs`, `MessageAvailabilityChecker.cs`, `RunDiagnostics.cs`, `GenerationPipeline.cs` and `GenerationLimits.cs` are byte-identical at the two commits. `BulkCreator.cs` line numbers are given as master / b0654bc.

Batch size

- Default 500: `GenerationLimits.DefaultBatchSize` (`src/SeedBomb.Core/Contracts/GenerationLimits.cs:15`), used by `GenerationConfig.BatchSize` (`GenerationConfig.cs:35`), `AppSettings.DefaultBatchSize` (`AppSettings.cs:21`) and `GenerateViewModel._batchSize` (`GenerateViewModel.cs:235`).
- Max 1000, min 1: `GenerationLimits.MaxBatchSize` (`GenerationLimits.cs:12`), enforced by `Validate` (`:33`), clamped in `GenerateViewModel.BuildConfig` (`GenerateViewModel.cs:873`) and `AppSettings.Clamped` (`AppSettings.cs:35`).
- Rows are chunked per table with `rowIndexes.Chunk(config.BatchSize)` (`BulkCreator.cs:371 / 374`). The same `BatchSize` is used for backfill and N:N batches.
- Record count: default 10 (`GenerationLimits.cs:21`), max 100,000 per table (`:18`). App retry count: `GenerationConfig.MaxRetries = 3` (`GenerationConfig.cs:45`). The UI never sets it.

DOP, step by step

1. UI value: `GenerateViewModel._maxParallelism` is an `int`, default 0 (`GenerateViewModel.cs:236`), loaded from `AppSettings.DefaultDop` (default 0, `AppSettings.cs:22`, clamped 0..16 at `:36`) at `GenerateViewModel.cs:181` and `:805`.
2. Empty box. Master: `Value="{Binding MaxParallelism}"` with no converter (`GeneratePage.xaml:277`). A cleared box most likely leaves the previous number in place (inference, see `50280b4`). b0654bc: the binding goes through `NullToZeroConverter` (`GeneratePage.xaml:277`), so an empty box gives 0.
3. `BuildConfig`: `MaxParallelism = MaxParallelism == 0 ? null : Math.Clamp(MaxParallelism, 1, GenerationLimits.MaxDop)` (`GenerateViewModel.cs:874`). `MaxDop = 16` (`GenerationLimits.cs:24`). `Validate` rejects values outside 1..16 (`GenerationLimits.cs:36`).
4. Warmup: `if (config.MaxParallelism is null && config.PlannedTotal > 0)` sends one `WhoAmIRequest` (`BulkCreator.cs:121 / 122`, call at `:770 / :786`). It runs once per run, after the lookup prep and pool reads and before the snapshot, watermarks and first table.
5. Per table, at the top of each table's iteration: `ResolveDop` reads `RecommendedDopHint = sc.RecommendedDegreesOfParallelism` (`BulkCreator.cs:744-745 / 747-748`). This is a property read, not a call. It is re-read for every table.
   - Master (`BulkCreator.cs:750-764`): explicit gives configured; else a hint `> 0` gives the hint, unclamped (logged when `> 16`); else `DefaultDop = 8` (`:32`).
   - b0654bc (`BulkCreator.cs:755-780`): explicit and (`hint <= 0` or `configured <= hint`) gives configured; explicit and `configured > hint` gives **the hint**, with a Warning log and a `Notices` entry; automatic is the same as master.

    - Fallback with no hint: 8 in the code. With a real `ServiceClient`, the SDK source (master branch on GitHub) initialises the value to 5, so the 8 is never reached (section 2.2; the 1.2.27 tag was not checked separately).
6. Applied as `ParallelOptions.MaxDegreeOfParallelism = effectiveDop` for that table's `Parallel.ForEachAsync` over batches (`BulkCreator.cs:399 / 402`). Backfill and N:N run their batches one after another (no DOP).

App retry and backoff (`src/SeedBomb.Bulk/ThrottlePolicy.cs`, identical in both)

- `maxRetries = config.MaxRetries = 3`, so up to 4 attempts (`ThrottlePolicy.cs:49`).
- Delay: `min(60 s, 2 s × 2^attempt) + uniform(-2000 ms, +2000 ms)` (`:125-131`, `BaseDelay` at `:18`). That gives about 2 s, 4 s and 8 s (each ±2 s), roughly 14 s in total, and is cancellable by the run token (`:70`).
- A caller cancel is rethrown immediately (`:57-60`).
- Retried (`RetryReason`, `:101-115`): `TimeoutException`; any `OperationCanceledException` not from the caller's token (including `TaskCanceledException` from an HttpClient timeout); `HttpRequestException`; `IOException`; `EndpointNotFoundException`; `ServerTooBusyException`; a `ProtocolException` whose message contains `"(429)"` (`:122-123`, **not** "(503)"); a `CommunicationException` (including `ProtocolException`) whose `InnerException` is `HttpRequestException` or `IOException` (`:117-118`).
- Not retried: `FaultException<OrganizationServiceFault>`, which is terminal and wrapped (`:72-79`), and everything else (`:80-85`).
- The SDK's own retries sit inside each attempt. `ServiceClient` is built without `MaxRetryCount` or `RetryPauseTime`, so the SDK defaults apply. `sb-investigate.md` gives them as 10 retries and 5 s, which section 2.2 confirms.

`ServiceClient` **construction** (`src/SeedBomb.Wpf/Services/Dataverse/DataverseConnectionService.cs:101-110`, identical in both)

- `new ServiceClient(instanceUrl, tokenProviderFunction: CreateTokenProvider(scopes), useUniqueInstance: true, logger: sdkLogger) { EnableAffinityCookie = false }`.
- `sdkLogger` is `null` unless Settings → Run diagnostics is on, in which case it is a Warning-filtered logger (`:49-64`, `:97`).
- Nothing sets `MaxRetryCount`, `RetryPauseTime` or `MaxConnectionTimeout` (no match anywhere in the repo).
- The token provider calls `_auth.GetTokenAsync(scopes, CancellationToken.None)` on every SDK token request (`:159-160`).
- The client is built once per profile and environment and then cached. Each run receives the cached client.

CreateMultiple to ExecuteMultiple fallback (`BulkCreator.SubmitBatchAsync`, `BulkCreator.cs:482-520 / 485-523`, identical logic)

- If `_messageChecker.ShouldAttemptCreateMultiple(entityName)` (in-memory; a new `MessageAvailabilityChecker` per run, `GenerationPipeline.cs:77`), send `CreateMultipleRequest` through `ThrottlePolicy`.
- If that throws `DataGenerationException` whose inner is `FaultException<OrganizationServiceFault>` with `ErrorCode == 0x80040800`: `MarkUnsupported`, so later batches of that table skip `CreateMultiple` for the rest of the run, then fall back (`:501-507 / :504-510`).

- If it throws `DataGenerationException` whose inner is any other `FaultException<OrganizationServiceFault>`: fall back for this batch only (`:508-514 / :511-517`).
- Fallback: one `ExecuteMultipleRequest` of the batch's `CreateRequest`s with `ContinueOnError = true` and `ReturnResponses = true` (`:599 / :602`), through `ThrottlePolicy`. Per-row faults become one `BatchError` per row.
- Exhausted transient retries, or any non-fault exception, do **not** fall back. They become one `BatchError` for the whole batch, with `IsTransient` set.

Dataverse calls a run makes other than creates (counts per run unless stated; identical at master and b0654bc)

Before the first write, in this order:

1. Metadata: one `RetrieveEntityRequest` (`EntityFilters.All`) **per selected table on a cache miss**, all issued at once by `Task.WhenAll` (`GenerationPipeline.cs:49` → `DataverseMetadataProvider.cs:74`, request at `:48-55`). The cache lasts 30 minutes (`:30`), so these are normally hits because the wizard already loaded them. Not through `ThrottlePolicy`.
2. Lookup candidate prep: `PreparedLookupRun.PrepareAsync` (`BulkCreator.cs:110 / 111`). For each distinct target of every `lookupRandom` column on a planned table, explicit or implicit (implicit ones are added by preflight for SystemRequired lookups): one `RetrieveEntityRequest` (`EntityFilters.Entity`) only if the target's metadata is not in the run (`PreparedLookupRun.cs:135-137`), plus 1 or 2 `RetrieveMultiple` pages of 500 rows, up to 1,000 per target (`:146-168`, `PageSize :24`, max `FieldRule.cs:89`). The reads run one after another, through `ThrottlePolicy`, including for `PreferInRun` columns.
3. `transactioncurrency`: one `RetrieveMultiple`, `TopCount = 10` (`BulkCreator.cs:688 / 691`). Direct; failures are logged.
4. `systemuser`: one `RetrieveMultiple`, `TopCount = 25`, `isdisabled = false` (`BulkCreator.cs:719 / 722`). Direct.
5. `WhoAmIRequest`: one, only when DOP is automatic and `PlannedTotal > 0` (`BulkCreator.cs:121 / 122`).
6. Reconcile watermarks (only with Run diagnostics on): one `RetrieveMultiple` (`TopCount = 1`, order by `modifiedon` desc, `createdby` = caller) per planned table (`RunDiagnostics.cs:54-81`; called at `BulkCreator.cs:129 / 130`).

Between tables: none. `ResolveDop` and `ShouldAttemptCreateMultiple` are in-memory.

After the last table's creates: 7. Deferred lookup backfill (`BulkCreator.cs:194 / 195`). These are writes, not creates. Per source table with deferred lookups: one `sdkmessagefilter RetrieveMultiple` to check `UpdateMultiple` support (`DeferredLookupBackfill.cs:136` → `MessageAvailabilityChecker.cs:72-91`, cached per run). Then `UpdateMultipleRequest` batches (`DeferredLookupBackfill.cs:278`) or `ExecuteMultiple` of `UpdateRequest` batches (`:318`), one after another, through `ThrottlePolicy`. 8. N:N associations (`BulkCreator.cs:203 / 204`): `ExecuteMultiple` batches of `AssociateRequest` (`DeferredLookupBackfill.cs:230`, `:318`), one after another, through `ThrottlePolicy`. 9. Reconcile count (only with Run diagnostics on): one aggregate FetchXML `count` per planned table (`RunDiagnostics.cs:92-160`; called at `BulkCreator.cs:213 / 214` on success and `:231 / 233` on the failed or cancelled path). Sequential, with a 30 s total timeout (`RunDiagnostics.cs:19`).

Not counted above: building `ServiceClient` (done once per connection, not per run).

Corrections to `sb-investigate.md` for this window

- Section 1 says the post-`00e6c41` commits "fix the DOP hint, the metadata cache, wrapped socket faults and logging. None of them touches the nested retry stack." That is wrong. `256faa9` adds wrapped-`CommunicationException` retries, and `35f4d23` makes `ExecuteAsync` retry `EndpointNotFoundException` and `ServerTooBusyException`, which were previously failed at once. Both widen the app-level retry layer. The timeout resend (IN-004) and the fallback-on-any-fault are indeed unchanged through `b0654bc`.
- F1 says `ThrottlePolicy` retries `ProtocolException` text containing "(429)" or "(503)". `IsRateLimited` matches only `"(429)"`, both at `00e6c41` and at `b0654bc`. A 503 can reach the app retry only as `ServerTooBusyException`, and only after `35f4d23` (the mapping is unverified).
- Sections 2 and F6 describe Oct 28 to Nov 1 as "on this branch". `256faa9` through `7e95da1` shipped in v1.0.1 and v1.0.2, and `449cac2` is on master. Only `50280b4`, `837d63b` and `b0654bc` are branch-only. F6's "Phase 3 (`7cb61b5`) and `837d63b` fix this on the branch" is wrong about `7cb61b5`, which has shipped.
- F6 treats Phase 3 only as a fix. It also lowers first-table DOP (and whole-run DOP where v1.0.0 never adopted the hint) whenever the hint is below 8, and its "else 8" fallback is probably unreachable with a real `ServiceClient` (SDK default 5, confirmed in section 2.2).
- The Phase 2 summary omits the opt-in reconcile queries. The pre-run watermark reads come from `7e95da1`, not Phase 2 (Recommendation 3 calls it "Phase 2's `modifiedon` watermark reconcile"). Section 4's "the Phase 2 commit message ... says 'Found in the first live run'" is `72fd797`'s message.
- F8's package list omits `Microsoft.Extensions.Caching.Memory` 10.0.12 → 10.0.14 (`695b42d`) and the `System.Security.Cryptography.Xml` 10.0.10 → 10.0.14 pin (`256faa9`). `Dataverse.Client` 1.2.27 is correct.
- The report does not mention `449cac2` (no backdated `createdon` from master on) or `50280b4` (an empty DOP box means automatic, branch only).

4.3 Window C: Nov 2 to Nov 11 2026 (b0654bc..a5fbbe5; branch fix/dop-cap-at-server-hint only, not in master or any tag)

Twenty commits, all made between 2026-11-02 19:40 and 2026-11-03 01:25 US Eastern: two docs commits (the investigation and the plan), sixteen code commits that implement plan Tasks 1 to 9 plus three follow-ups the plan does not have (`a2c6bcb`, `9613a9b`, `a5fbbe5`), and two docs commits for Task 10. There are **no commits after Nov 3** on any ref except the unmerged dependabot branch `origin/dependabot/nuget/nuget-77c1b23c9f` (`a68cee6`, Nov 11, NuGet bumps). `fix/dop-cap-at-server-hint`, `origin/fix/dop-cap-at-server-hint` and HEAD are all `a5fbbe5`. No `pre-<topic>` tag was created, even though Task 10's checklist asks for one. The window does not touch DOP resolution, `GenerationConfig.MaxRetries` (3) or the default batch size (500). What it changes is who retries and how long it waits, a run-wide `Retry-After` pause, no `ExecuteMultiple` replay of data faults, no resend of a create after a timeout or network fault, adaptive batch size from 250 rows, and fewer pre-run lookup reads. In the healthy steady state almost all of this is neutral; the measurable healthy-state costs are the 250-row start (reset for every table) and that a batch only grows after a call of at most 20 s. Several changes cut **rows written** rather than requests per second: a data fault now rejects the whole batch, and a timeout or network fault on an unkeyed table leaves the batch as "outcome unknown".

Notation: `attempt` is 0-based. `MaxRetries` = 3 everywhere in production (`GenerationConfig.cs:46`; nothing in `SeedBomb.Wpf` sets it), so 4 attempts at most.

`26fa1fc` · **2026-11-03 00:40 +0000 · Add investigation of the generation timeouts and HTTP error flood since late September**

- **Plan task / recommendation:** source of recommendations 1 to 9 (section 5).
- **Area:** docs
- **Files:** `sb-investigate.md` (+191)
- **Before → After:** none in code. Findings F1 to F8. It attributes the stalls to nested SDK (10) and app (3) retries, timeout resends (IN-004) and the `ExecuteMultiple` fallback on any `FaultException`. It states SDK defaults it read but did not run: `MaxRetryCount = 10`, `RetryPauseTime = 5 s`, `MaxConnectionTimeout = 4 min`.
- **Throughput effect:** `neutral`. Docs only.
- **Deviation from plan:** n/a.
- **Notes:** Recommendation 4 asks for the gate on "the whole table's `Parallel.ForEachAsync`"; the plan made it run-wide (D3). Recommendation 3 asks to split timed-out batches generally; the plan restricts splitting to tables with an alternate key (D2).

`936e754` · **2026-11-03 01:06 +0000 · Plan the write-path retry ownership work from the investigation's recommendations**

- **Plan task / recommendation:** the plan itself (Tasks 1 to 10, decisions D1 to D5).
- **Area:** docs
- **Files:** `docs/plans/11-03-2026-write-path-retry-ownership.md` (+1789; unchanged at HEAD)
- **Before → After:** none in code. Key decisions: D1 SDK `MaxRetryCount = 0`, app owns retries. D2 timed-out create is split only on keyed tables, otherwise `OutcomeUnknown`. D3 the pause is run-wide, inside `ThrottlePolicy`. D4 `Retry-After` is not capped. D5 adaptive size `min(configured, 250)`, ×1.5 after a call ≤ 20 s, halve (floor 25) on timeout.
- **Throughput effect:** `neutral`. Docs only.
- **Deviation from plan:** n/a.
- **Notes:** The plan says none of its `dotnet` commands were run (no SDK in its container).

`1b5ef40` · **2026-11-02 20:28 -0500 · Make ThrottlePolicy the only retry owner and honour Retry-After**

- **Plan task / recommendation:** Task 1; recommendations 1 and 8.
- **Area:** retry, throttle+Retry-After, client config
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `src/SeedBomb.Wpf/Services/Dataverse/DataverseConnectionService.cs`, `src/SeedBomb.Bulk.Tests/ThrottlePolicyTests.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`
- **Before → After:**
  - `ServiceClient` initializer: before, it set only `EnableAffinityCookie = false`, so SDK defaults applied (investigation: 10 retries, 5 s pause). After: `MaxRetryCount = 0`, `RetryPauseTime = TimeSpan.Zero`, `EnableAffinityCookie = false`.

    - `RetryReason` (the single classifier for both retrying and `IsTransient`) gains two classes:
        - `FaultException<OrganizationServiceFault> f when IsServiceProtection(f)` → "Service protection", for codes `[-2147015902, -2147015903, -2147015898]`.
        - `ProtocolException p when IsGatewayFault(p)` → "HTTP 502/503": the message contains `"(502)"` or `"(503)"`.
    - Already retried before this commit: `TimeoutException`, `OperationCanceledException` (not the caller's own cancel), `HttpRequestException`, `IOException`, `EndpointNotFoundException`, `ServerTooBusyException`, `ProtocolException` containing `"(429)"`, and `CommunicationException` wrapping `HttpRequestException`/`IOException`.
    - Delay: `var delay = RetryAfterOf(ex) ?? ComputeDelay(attempt);`. `RetryAfterOf` reads `fault.Detail.ErrorDetails["Retry-After"]` only when it is a `TimeSpan` greater than zero, and applies no cap or floor. `ComputeDelay` is unchanged: `min(2 s × 2^attempt, 60 s) + U(−2000, +2000) ms` (additive uniform jitter from `Random.Shared`). With `MaxRetries` 3 that gives waits in [0, 4) s, [2, 6) s and [6, 10) s. A service-protection fault **without** a `Retry-After` detail, and a 429 that arrives as `ProtocolException`, use that schedule, so the first resend can happen almost immediately.
    - A service-protection fault is now transient, so `BatchError.IsTransient` is true and the sheet offers it as "Retryable".
- **Throughput effect:** `unclear`.
    - Healthy: neutral; no faults means no retries.
    - Throttled: the nested 11 × 4 SDK×app loop is gone, which removes multi-minute silent stalls (raises wall-clock progress). But each batch now gets 4 attempts in total instead of up to 11 SDK attempts inside each app attempt, so more throttled batches end as rejected rows (fewer rows written).
    - Inference: calls that do not go through `ThrottlePolicy` lost their SDK retries entirely: the WhoAmI warmup (`BulkCreator.cs:889`), currency and systemuser pool reads (`:796`, `:829`), watermark and reconcile queries (`RunDiagnostics.cs:78`, `:163`), `MessageAvailabilityChecker` (`:91`), and the metadata provider (`DataverseMetadataProvider.cs:55`, `:114`). If the warmup fails, the hint is whatever an earlier response in the run set, or the SDK's default of 5 (section 2.2); the fallback to 8 is not reached with a real client.
    - Checked afterwards in the SDK's public source (section 2.2): with `MaxRetryCount = 0` the SDK rethrows the exception it caught, so a throttle that arrives as a SOAP fault reaches the app as a `FaultException` with `ErrorDetails["Retry-After"]` intact. If it surfaces as `ProtocolException (429)` instead (the investigation says the SDK does this on some paths), the server's wait is ignored and the 0 to 10 s backoff applies.
- **Deviation from plan:** none found; the code matches Task 1 Step 3 line for line. A gap in the plan itself: D1 says "every fault reaches this policy", but the calls listed above never pass through it.
- **Notes:** Tests:
    - `ExecuteAsync_ThrottleFaultWithRetryAfter_WaitsOnceAndStaysWithinBudget`: 3 codes, `maxRetries: 2`, 300 ms Retry-After. Asserts exactly 3 calls, elapsed 600 ms to 2 s, and transient.
    - `..._WithoutRetryAfter_UsesBackoff`: 2 calls.
    - `ExecuteAsync_ThrottleWait_IsCancellable`: a 5 min Retry-After is cancelled after 100 ms, inside 5 s.
    - `ExecuteAsync_ThrottleFault_IsRetriedByPolicy`: `maxRetries: 1` gives 2 calls.
    - `ExecuteAsync_GatewayProtocolFault_IsRetried`: 502 and 503, 2 calls each.

- `TransientCases`: the (502) row flips to true; a -2147015902 fault row is added as true.
- `CreateAsync_WholeBatchFault_KeepsTheDataverseFaultCode`: now expects `IsTransient` true.
- Amended by `aea75f5` (docs), `e35635e` (pause), `5567cf6` and `a5fbbe5` (resend flags), and `5947b70` (settings replace the literal values).

`aea75f5` · 2026-11-02 20:33 -0500 · Correct stale retry-ownership docs after Task 1

- **Plan task / recommendation:** Task 1 follow-up (not a plan step).
- **Area:** docs
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs` (XML docs only), `src/SeedBomb.Bulk.Tests/DeferredLookupBackfillTests.cs` (comment only)
- **Before → After:** XML docs for `ExecuteAsync`, `IsTransient` and `RetryAfterOf` rewritten. `RetryAfterOf` is documented as not checking that the fault is a service-protection fault. No code change.
- **Throughput effect:** `neutral`.
- **Deviation from plan:** none found (not in the plan).
- **Notes:** none.

`e35635e` · 2026-11-02 20:38 -0500 · Pause every in-flight batch for one shared Retry-After

- **Plan task / recommendation:** Task 2; recommendation 4 (second half).
- **Area:** shared pause, throttle+Retry-After
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `src/SeedBomb.Bulk.Tests/ThrottlePolicyTests.cs`
- **Before → After:**
  - Data structure: `private long _pausedUntil;`, a `Stopwatch` timestamp where 0 means never paused. It is per `ThrottlePolicy` instance, and `GenerationPipeline.cs:78` builds one per run. That instance is shared by every table's creates, `PreparedLookupRun` reads, the per-table fallback reads (`9613a9b`), deferred-lookup backfill and N:N association. The lookup-picker UI uses a different instance (the DI singleton, `App.xaml.cs:228`).
  - Who sets it: only the transient `catch`, and only when `attempt < maxRetries` and `RetryAfterOf(ex)` is non-null: `if (retryAfter is { } serverWait) PauseFor(serverWait);`. `PauseFor` computes `now + wait` and only ever extends, using a CAS loop (`if (until <= seen) return;`).
  - Who waits: every `ExecuteAsync` attempt calls `await WaitIfPausedAsync(ct)` right after `ct.ThrowIfCancellationRequested()`, before the operation runs. In the batch loop that point is before each `CreateMultiple` or `ExecuteMultiple` attempt, including retries and split halves. A call already in flight is not interrupted.
  - The faulting caller first sleeps its own `delay` (= its Retry-After) with `Task.Delay(delay, ct)`, then goes through `WaitIfPausedAsync` on its next attempt.
  - Waiters sit inside the `Parallel.ForEachAsync` body, so they keep their DOP worker. There are no free slots during a pause; that is the intended effect.

    - Clearing: never; a deadline in the past is simply a no-op. Cancel works through `Task.Delay(..., ct)`.
    - Scope: one table's throttle pauses every table, the reads and backfill for the rest of the run. Calls that bypass `ThrottlePolicy` (pool reads, WhoAmI, watermark and reconcile, metadata) are not paused.
- **Throughput effect:** `lowers` in the throttled state, by design: concurrency drops to 0 for the longest `Retry-After` seen, run-wide. Healthy: neutral (one `Interlocked.Read` per attempt). Inference: in the throttled state it may raise net rows per second, because it avoids the thundering herd that grows `Retry-After`.
- **Deviation from plan:** none found (matches Task 2 Step 3). The investigation asked for a per-table pause; the plan's D3 and this code are run-wide.
- **Notes:** Test `ExecuteAsync_ConcurrentCallers_ShareOneRetryAfterPause`: a 500 ms Retry-After on caller 1 makes caller 2 (which never faulted) take 350 ms to 2 s. A side effect: on the **final** attempt a `Retry-After` arms nothing, and runs with `MaxRetries = 0` never pause. Amended by `a2c6bcb`.

`a2c6bcb` · **2026-11-02 20:49 -0500 · Re-check the pause deadline so a longer Retry-After reaches waiters**

- **Plan task / recommendation:** Task 2 follow-up (not in the plan); recommendation 4.
- **Area:** shared pause
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `src/SeedBomb.Bulk.Tests/ThrottlePolicyTests.cs`
- **Before → After:** `WaitIfPausedAsync` read the deadline once and slept. It is now a `while (true)` loop that re-reads `_pausedUntil` after each `Task.Delay`. A waiter therefore keeps sleeping when another caller extends the deadline mid-wait, and returns only once `remaining <= 0`.
- **Throughput effect:** `lowers` (throttled only). A waiter now sits out every extension instead of waking at the first deadline. Healthy: neutral.
- **Deviation from plan:** Yes. The plan's `WaitIfPausedAsync` was single-shot; this makes it a loop.
- **Notes:** Tests:
    - `ExecuteAsync_ShorterLaterRetryAfter_DoesNotShortenTheArmedPause`: 700 ms is armed, then 150 ms arrives; a third caller waits 400 ms to 3 s.
    - `ExecuteAsync_LongerLaterRetryAfter_ExtendsAnAlreadyWaitingCaller`: 300 ms is armed, then 800 ms arrives; the waiter takes 550 ms to 3 s.

`8b50639` · **2026-11-02 20:53 -0500 · Show a service-protection wait on the run sheet**

- **Plan task / recommendation:** Task 3; recommendation 1 ("surface the wait").
- **Area:** progress+logging, shared pause
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Wpf/Services/Generation/ProgressUpdate.cs`, `src/SeedBomb.Wpf/Services/Generation/WpfGenerationService.cs`, `src/SeedBomb.Wpf/ViewModels/RunViewModel.cs`, `src/SeedBomb.Wpf.Tests/RunViewModelTests.cs`
- **Before → After:**
    - New `public event Action<string, TimeSpan>? Waiting`, raised just before `Task.Delay(delay, ct)` in the transient catch. It fires for **every** retry class (timeout, network, 429, 502/503, service protection), not only throttles. Callers that are merely waiting on the shared pause do not raise it.

    - `BulkCreator.CreateAsync` subscribes `ReportWait` for the whole run and unsubscribes in `finally`. `ReportWait` reports `Phase = "Waiting"` with the text `$"Server busy, resuming in {Math.Ceiling(delay.TotalSeconds):F0} s ({entity})"`.
    - `ProgressUpdate` gains `string? Note = null`. `WpfGenerationService` now passes `p.ErrorMessage` as `Note` on every snapshot.
    - `RunViewModel.AcceptProgress`: a "Waiting" snapshot sets `StatusHeadline` to the note (unless it is "Cancelling...") and appends an activity line; counts are not touched.
    - Phase-only snapshots bypass the 100 ms `ProgressGate`.
- **Throughput effect:** `neutral`. One UI report per retry sleep.
- **Deviation from plan:** none found (matches Task 3). The "Server busy" wording also labels network and timeout retries, which the plan did not flag.
- **Notes:** Test `AcceptProgress_WaitingSnapshot_ShowsTheServerWaitWithoutTouchingCounts`: headline equals the note, and `RowsWrittenLabel` stays "40". Amended by `a5fbbe5` (a waiting snapshot that arrives after "Cancelled" is ignored).

`c8a8b0f` · **2026-11-02 20:57 -0500 · Fall back to ExecuteMultiple only when CreateMultiple is unsupported**

- **Plan task / recommendation:** Task 4; recommendation 2.
- **Area:** fallback, batch submission
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`
- **Before → After:**
    - Deleted from `SubmitBatchAsync`: `catch (DataGenerationException ex) when (ex.InnerException is FaultException<OrganizationServiceFault>)`, which fell through to a full `ExecuteMultiple` (one `CreateRequest` per row, `ContinueOnError = true`) with its own `MaxRetries`.
    - The only fallback left is `IsCreateMultipleUnsupported`: `fault.Detail?.ErrorCode == unchecked((int)0x80040800)`. It calls `_messageChecker.MarkUnsupported(entityName)`, so later batches of that table go straight to `ExecuteMultiple`.
    - Every other `DataGenerationException` (data fault, plug-in, duplicate key, an exhausted service-protection fault, or a 429/502/503 after the last attempt) now propagates to the per-batch `catch` in `SubmitEntityBatchesAsync`. That catch records one `BatchError` with `FaultCode`, `RowCount = batch.Length`, `RowIndexes = rows` and `IsTransient = ThrottlePolicy.IsTransient(inner)`.
- **Throughput effect:** `unclear`.
    - Requests: raises. A faulting batch no longer costs up to 4 more `ExecuteMultiple` sends of N sub-requests each.
    - Rows written: lowers whenever some rows in a batch fault. `CreateMultiple` on a standard table is all-or-nothing, so before, one bad row in 500 led to `ExecuteMultiple` writing the other 499; now all 500 are rejected.
    - Inference: on data where a small fraction of rows fault (plug-ins, required fields, a duplicate key), rows created per second can fall by close to the whole batch-failure rate. Healthy data: neutral.
- **Deviation from plan:** none found (matches Task 4, including the class summary text).
- **Notes:** Test `CreateAsync_DataFaultOnCreateMultiple_RecordsOneBatchErrorAndNeverFallsBack`: error `-2147220969`, 10 rows in one `BatchError`, `RowIndexes` 0 to 9, `ExecuteMultipleRequest` never sent.

`5567cf6` · 2026-11-02 21:03 -0500 · Split or report a timed-out create instead of resending it

- **Plan task / recommendation:** Task 5; recommendation 3.
- **Area:** timeout+cancel, retry, batch size
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `src/SeedBomb.Core/Contracts/BatchError.cs`, `src/SeedBomb.Wpf/ViewModels/RunViewModel.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`, `src/SeedBomb.Bulk.Tests/ThrottlePolicyTests.cs`
- **Before → After:**
  - `ThrottlePolicy.ExecuteAsync(..., bool resendOnTimeout = true)`. The retry filter adds `&& (resendOnTimeout || ex is not (TimeoutException or OperationCanceledException))`, so with `false` a timeout is terminal on attempt 0.
  - `CreateMultiple` and `ExecuteMultiple` in `SubmitBatchAsync` now pass `resendOnTimeout: false`. Reads, backfill and associate keep `true`.
  - A new catch on the `CreateMultiple` path (`IsClientTimeout`: inner `TimeoutException` or `OperationCanceledException` and the run token not cancelled) calls `ResolveTimedOutBatchAsync`:
    - When `hasAlternateKey && batch.Length > 1`: `half = batch.Length / 2`, then `SubmitBatchAsync(left)` followed by `SubmitBatchAsync(right)`. The halves run **sequentially** in the same worker, with the full `maxRetries` each, and recursively: a half that times out is split again, down to 1 row.
    - Otherwise: one `BatchError` with message `"Timed out after {s} s; outcome unknown. …"`, `IsTransient: false`, `OutcomeUnknown = true`, `RowIndexes = rows`. Nothing is resent.
  - `BatchError.OutcomeUnknown` added. `RunViewModel.BuildGroup`: unknown gives disposition "Unknown", not retryable, with a "check the table" hint.
  - Before this commit (state at `b0654bc`): a timeout was resent up to 3 times, each attempt able to run to the client timeout (4 min per the investigation), then recorded as a transient "Retryable" error.
  - Split cost: one timed-out call (up to the client timeout) plus at least 2 calls. Worst case with persistent timeouts is 2N−1 calls for N rows, each able to run to the timeout, all in one worker.
- **Throughput effect:** `unclear`.
  - Healthy: neutral.
  - Slow server: raises wall-clock progress, since there are no more 3 × 4 min resends.
  - Rows: lowers rows written on unkeyed tables, where a timed-out batch is now never written by this run.
  - Keyed tables: adds requests, run sequentially. Inference: if the original batch had committed, both halves fail as duplicate keys, so the rows are reported as rejected ("Duplicate key — fix the source data …") although they exist, and their IDs never enter the pool.
- **Deviation from plan:** none found (matches Task 5). The `ExecuteMultiple` path only gained `resendOnTimeout: false` with no split or unknown handling, so a fallback timeout became a transient "Retryable" error. `0cfb873` closes that.
- **Notes:** Tests:
  - `ExecuteAsync_TimeoutWithResendDisabled_IsTerminalAndStillTransient`: 1 call, inner exception is `TaskCanceledException`.
  - `CreateAsync_ClientTimeoutOnUnkeyedTable_ReportsUnknownOutcomeWithoutResending`: 1 send, `OutcomeUnknown`, not transient, 10 rows.

    - `CreateAsync_ClientTimeoutOnKeyedTable_SplitsTheBatchOnce`: sizes `[10, 5, 5]`. The name says "once", but the code recurses without limit.
    - Amended by `0cfb873`, `938b745`, `3d72627` and `a5fbbe5`.

`0cfb873` · 2026-11-02 21:12 -0500 · Report ExecuteMultiple timeouts as unknown or split like CreateMultiple

- **Plan task / recommendation:** extension of Task 5 (labelled "R6" in tests); recommendation 3.
- **Area:** timeout+cancel, fallback
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Wpf/ViewModels/RejectionGroup.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`, `src/SeedBomb.Wpf.Tests/RunViewModelTests.cs`
- **Before → After:** The `ExecuteMultiple` call in `SubmitBatchAsync` is wrapped in `try … catch (DataGenerationException ex) when (IsClientTimeout(ex, ct))`, which goes to the same `ResolveTimedOutBatchAsync` (split if keyed and longer than 1 row, otherwise unknown). Log text changes to "The create batch for {Entity} timed out ... ({TimeoutType})". `RejectionGroup` docs now list "Unknown".
- **Throughput effect:** `neutral` when healthy. With a slow server on the fallback path, the same trade as `5567cf6`: no resend, fewer rows written on unkeyed tables. `ExecuteMultiple` commits row by row, so a timed-out fallback batch may be partly written.
- **Deviation from plan:** Yes. This is beyond Task 5, which handled only the `CreateMultiple` path.
- **Notes:** Tests:
    - `CreateAsync_ClientTimeoutOnUnkeyedFallback_ReportsUnknownOutcomeWithoutResending`: 1 send, unknown.
    - `CreateAsync_ClientTimeoutOnKeyedFallback_SplitsTheBatchOnce`: sizes `[10, 5, 5]`.
    - `ApplyResult_UnknownOutcome_IsDispositionUnknownWithCheckTheTableHint`.

`938b745` · 2026-11-02 21:25 -0500 · Size CreateMultiple batches adaptively from 250 rows

- **Plan task / recommendation:** Task 6; recommendation 4 (first half).
- **Area:** batch size, batch submission, progress+logging
- **Files:** `src/SeedBomb.Bulk/AdaptiveBatchSize.cs` (new), `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk/RunDiagnostics.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`
- **Before → After:**
    - Before: `rowIndexes.Chunk(config.BatchSize)` gave fixed batches of the configured size (default 500); `Parallel.ForEachAsync` ran over the batch indexes.
    - `AdaptiveBatchSize(int configured)`:
        - `InitialSize = 250`, `FastCall = 20 s`, `Floor = 25`.
        - Start: `Math.Max(1, Math.Min(configured, 250))`.
        - `ObserveCall(d)`: when `d <= 20 s`, `_current = min(configured, _current + _current/2)`, so 250 → 375 → 500 at the default. A slower call leaves the size unchanged; it never shrinks.

        - `ObserveTimeout()`: `_current = max(min(25, configured), _current / 2)`.
        - Every access is under a `lock`, so it is thread-safe. It can grow back after shrinking.
        - The configured size is a **ceiling**, never exceeded. Configured sizes of 250 or less behave exactly as before.
    - State is per table: a new sizer in each `SubmitEntityBatchesAsync`, shared by that table's parallel workers. Every table restarts at 250.
    - Partitioning:
        - `Enumerable.Range(0, min(DOP, max(1, planned)))` workers each loop on `NextSlice()`. Under `cursorLock` it takes `min(sizer.Current, remaining)` contiguous row indexes from a shared cursor and assigns `allocatedBatches++` as the batch index.
        - The size is read when a slice is taken, so the first DOP slices are all 250 before any call returns.
        - Results go into `ConcurrentDictionary` bags and are merged in index order.
    - Per batch, after submit:
        - `ObserveCall(trace.LastCallMs)` when the batch has no `failureType` and no `OutcomeUnknown` errors. `LastCallMs` is the last attempt only; retry and pause waits are excluded.
        - `ObserveTimeout()` when the batch has `OutcomeUnknown` errors.
        - `ResolveTimedOutBatchAsync` also calls `ObserveTimeout()` before splitting a keyed batch.
    - The sizer also governs `ExecuteMultiple` fallback batches, not only `CreateMultiple`.
    - `TotalBatches` is now an estimate, `allocated + ceil(remaining / Current)`.
    - Snapshot line adds `batch N (starting at min(N, 250))`.
- **Throughput effect:** `lowers` slightly in the healthy state (inference).
    - Each table begins with DOP × 250-row calls. At the default 500 it reaches 500 after two fast completions.
    - If 250-row calls take more than 20 s (plug-ins, wide rows), the table stays at 250 for its whole length, so half the configured rows per request.
    - With call time roughly proportional to rows and DOP fixed, rows per second change only by per-request overhead. The effect should be small unless that overhead is large.
    - For small tables it can raise concurrency: 1,000 rows at DOP 8 now go out as four concurrent 250-row calls instead of two 500-row calls.
- **Deviation from plan:**
    - Batch index: the plan used `completedBatches + idBags.Count`; the code uses a separate `allocatedBatches++`, which is a fix.
    - Growth after a split: the plan grows on any batch that ends without errors, so a successful split lets it grow again after shrinking. `3d72627` changes this.
- **Notes:** Tests:
    - `AdaptiveBatchSize_StartsLowGrowsOnFastCallsAndHalvesOnTimeout`: 1000 configured gives 250; after a 5 s call 375; after a 25 s call still 375; 10 × 1 s calls give 1000; a timeout gives 500; 10 timeouts give 25; configured 10 gives 10.
    - `CreateAsync_AdaptiveBatchSize_StartsAt250AndGrowsTowardConfigured`: sizes[0] = 250, sizes[1] = 375, sum 1000, every size in [1, 500].
    - `..._ParallelWorkersSubmitEveryRowOnceWithUniqueBatchIndexes`: 2,000 rows, DOP 4.

`3d72627` · 2026-11-02 21:33 -0500 · Reduce adaptive batch size on every timeout

- **Plan task / recommendation:** Task 6 follow-up (not in the plan).
- **Area:** batch size, timeout+cancel
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`
- **Before → After:**
  - `ObserveTimeout()` moves out of the per-batch block and out of `ResolveTimedOutBatchAsync`, into both timeout `catch` blocks of `SubmitBatchAsync` together with a new `trace.MarkTimeout()`. It now fires once per timed-out **call**, including every level of a recursive split.
  - Growth is suppressed for any batch that had a timeout: `if (!trace.HadTimeout && failureType is null && …) sizer.ObserveCall(...)`.
  - Because the sizer is shared, N concurrent timeouts halve it N times: from 500, five timeouts reach the floor of 25. Recovery from 25 to 500 takes eight calls of 20 s or less.
- **Throughput effect:** `lowers` in the degraded state. A wave of timeouts at DOP 8 collapses the size to 25 rows, about 20× more requests per row, until fast calls rebuild it. Inference: under execution-time throttling, 25-row calls should finish fast and rebuild quickly; under request-count throttling they spend the budget faster. Healthy: neutral.
- **Deviation from plan:** Yes. The plan halves only for an `OutcomeUnknown` batch or before a split (once per batch), and lets a successful split grow the size again.
- **Notes:** Tests:
  - `CreateAsync_ClientTimeoutOnUnkeyedBatch_HalvesTheNextBatchSize`: 1,000 rows at batch 500; sizes[0] = 250 (times out), sizes[1] = 125, one error of 250 rows.
  - `CreateAsync_ClientTimeoutOnKeyedBatch_DoesNotGrowAfterSuccessfulSplits`: the first four sizes are `[250, 125, 125, 125]`.
  - Amended by `a5fbbe5`, which also shrinks on network faults.

`5dfc3aa` · 2026-11-02 23:07 -0500 · Skip lookup reads for records created in the run

- **Plan task / recommendation:** Task 7; recommendation 5.
- **Area:** pre-run reads, lookups, progress+logging
- **Files:** `src/SeedBomb.Bulk/PreparedLookupRun.cs`, `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Wpf/ViewModels/RunViewModel.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`, `src/SeedBomb.Wpf.Tests/RunViewModelTests.cs`
- **Before → After:**
  - `PrepareAsync` read every target of every `lookupRandom` column before the first write: at most 1,000 rows (`MaximumCandidatesPerTarget`), in pages of 500, plus a `RetrieveEntityRequest` for a target missing from the metadata set.
  - It now sets `readsEnvironment = !(preferInRunColumn && targets.All(t => inRunTargets.Contains(t)))`.
    - `preferInRunColumn` holds only for implicit required-lookup rules added by preflight; explicit rules always read.
    - `inRunTargets` are targets with `PlannedRows > 0` that sit strictly earlier in topological order.

    - Reads are skipped only when every target of the column qualifies. A polymorphic lookup where any target is not created earlier still reads all of them.
    - A `"Preparing lookups"` phase-only progress report is sent before `PrepareAsync`. `RunViewModel` sets the headline to `"{Phase}…"` for snapshots with no entity.
- **Throughput effect:** `raises`. Fewer pre-run `RetrieveMultiple` pages, each going through the policy. It only affects time before the first write, not rows per second during writes.
- **Deviation from plan:** none found (matches Task 7).
- **Notes:** Tests:
    - `CreateAsync_ImplicitRequiredLookupWithInRunParent_ReadsNothingFromTheEnvironment`: `RetrieveMultipleAsync` on account is never called, and the "Preparing lookups" phase is seen.
    - `AcceptProgress_PhaseOnlySnapshot_UsesThePhaseAsTheHeadline`.
    - Amended by `9613a9b`.

`9613a9b` · **2026-11-03 00:55 -0500 · Preserve lookup fallback after failed parent writes**

- **Plan task / recommendation:** Task 7 follow-up (not in the plan).
- **Area:** lookups, pre-run reads
- **Files:** `src/SeedBomb.Bulk/PreparedLookupRun.cs`, `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`
- **Before → After:**
    - A column whose read was skipped stores `DeferredTargets = targets`.
    - New `PrepareFallbackAsync(table, pool, …)` runs at the top of every table in the write loop (`BulkCreator.cs:179`). For each `PreferInRun` column of that table that has deferred targets, it reads those targets through `ThrottlePolicy` (same `ReadTargetAsync`, cached per target) **only if** none of its `InRunTargets` has any rows in the pool, meaning the parents wrote nothing.
    - `Get` uses `_deferredExisting` when there are no in-run candidates.
- **Throughput effect:** `neutral` in the healthy state (no reads when parents were written). Degraded: it adds reads only after a parent table wrote zero rows.
- **Deviation from plan:** Yes. This is beyond Task 7, which kept only the "neither source → throw" check.
- **Notes:** Test `CreateAsync_ImplicitRequiredLookupWithFailedInRunParent_FallsBackToEnvironment`: the account create fails, so account is read exactly once and the contact links to the existing account.

`1a62a2e` · **2026-11-03 01:01 -0500 · Collapse repeated activity lines and summarise identical batch failures**

- **Plan task / recommendation:** Task 8; recommendation 6.
- **Area:** progress+logging

- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Wpf/ViewModels/RunViewModel.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`, `src/SeedBomb.Wpf.Tests/RunViewModelTests.cs`
- **Before → After:**
  - The per-batch failure log changes from `LogError(ex, "Batch {i}/{n} for {Entity} failed")` (exception with stack) to `LogWarning("... failed: {Message}")` (message only).
  - After the table, one `LogError` per distinct message for errors with more than 1 row index: `"{Entity}: {Batches} batch(es) ({Rows} rows) failed with: {Message}"`.
  - `AppendActivity` collapses identical consecutive lines to `"line (×N)"` via `_lastActivityRepeats`, which is reset in `StartRun`.
  - Unchanged per batch: one `[timing] batch` Information line, the `ExecuteMultiple` per-batch warning (`BulkCreator.cs:733`), each `ThrottlePolicy` retry warning, and a full rebuild of `RecentActivity` (last 4) on every activity line.
- **Throughput effect:** `neutral`. Slightly less logging work per failed batch.
- **Deviation from plan:** none found (matches Task 8).
- **Notes:** Tests:
  - `ApplyResult_CollapsesIdenticalConsecutiveNoticesWithACount`: "Server busy (×3)".
  - `CreateAsync_IdenticalBatchFailures_AreSummarisedOncePerTable`: "account: 3 batch(es) (30 rows)".
  - Waiting notes carry a changing second count and the entity name, so consecutive waits rarely collapse.
  - Amended by `f51a23f`.

`f51a23f` · 2026-11-03 01:06 -0500 · Include fallback item faults in batch failure summary

- **Plan task / recommendation:** Task 8 follow-up (not in the plan).
- **Area:** progress+logging
- **Files:** `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`
- **Before → After:** The summary now groups every error in `errorBags`, including single-row `ExecuteMultiple` item faults, by `ErrorMessage`. `Batches` is the count of distinct batch indexes and `Rows` is the sum of `RowCount`. Before, only errors with more than 1 row index were summarised.
- **Throughput effect:** `neutral`.
- **Deviation from plan:** Yes. It drops the plan's `RowIndexes.Count > 1` filter and counts distinct batches.
- **Notes:** The test is rewritten to use `0x80040800` fallback with 2 faulting items per batch of 10: 6 errors, rows `[0,1,10,11,20,21]`, "account: 3 batch(es) (6 rows) failed with: Attribute validation error".

`5947b70` · 2026-11-03 01:11 -0500 · Expose Dataverse client retry and timeout settings

- **Plan task / recommendation:** Task 9; recommendation 7.
- **Area:** settings+limits, client config, progress+logging

- **Files:** `src/SeedBomb.Wpf/Services/Settings/AppSettings.cs`, `src/SeedBomb.Wpf/Services/Dataverse/DataverseConnectionService.cs`, `src/SeedBomb.Wpf/ViewModels/SettingsViewModel.cs`, `src/SeedBomb.Wpf/Views/Pages/SettingsPage.xaml`, `src/SeedBomb.Bulk/RunDiagnostics.cs`, `src/SeedBomb.Wpf.Tests/DataverseConnectionServiceTests.cs`, `src/SeedBomb.Wpf.Tests/SettingsViewModelTests.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`
- **Before → After:**
  - New `AppSettings` parameters:

Setting	Default	Clamp
`SdkMaxRetryCount`	0	0–10
`SdkRetryPauseSeconds`	0	0–300
`ConnectionTimeoutMinutes`	4	1–30

    Settings shows them as `NumberBox` controls under "Advanced: Dataverse client".

  - `ClientOptionsAsync(settings)` loads and clamps the settings, and falls back to `AppSettings.Default` on any load failure. In `GetOrganizationServiceAsync`, before constructing the client, it sets `ServiceClient.MaxConnectionTimeout = options.ConnectionTimeout`. That is a **static**, read at construction. The initializer becomes `MaxRetryCount = options.MaxRetryCount, RetryPauseTime = options.RetryPauseTime, EnableAffinityCookie = false`. Changes apply to the next connection only.
  - Before: `MaxConnectionTimeout` was never set, so the SDK default applied (4 min, confirmed in section 2.2). The app also now overrides any `MaxDataverseConnectionTimeOutMinutes` app setting.
  - The snapshot line adds `SDK retries {n}, SDK retry pause {t}, connection timeout {static}`.
- **Throughput effect:** `neutral` at the defaults, which reproduce `1b5ef40`'s literals and the investigation's 4 min. `unclear` if a user raises `SdkMaxRetryCount`, which reintroduces nested retries under the app's 3.
  - Inference: an existing `settings.json` without the new keys should deserialize with the constructor defaults (System.Text.Json honours optional parameter defaults), giving 4 min. No test covers loading an older file. If the keys came back as 0, `Clamped()` would make the timeout **1 minute**, which would turn many large creates into "outcome unknown".
- **Deviation from plan:** none found (matches Task 9; the settings test also loads non-default values first).
- **Notes:** Tests:
  - `ClientOptions_ComeFromSettings`: 2 retries, 7 s, 6 min.
  - `ClientOptions_DefaultToAppOwnedRetries_WhenSettingsAreUnavailable`: 0, 0, 4 min.
  - `SaveAsync_PersistsTheDataverseClientKnobs`.

- The snapshot test asserts `"SDK retries n/a, SDK retry pause n/a, connection timeout "`.

`120f5f5` · **2026-11-03 01:15 -0500 · Document the live smoke required before merging a write-path change**

- **Plan task / recommendation:** Task 10; recommendation 9.
- **Area:** docs
- **Files:** `documentation/how-to/developer/diagnostics.md`
- **Before → After:** Adds "## 5. Before merging a write-path change": tag first, turn on Run diagnostics, run a few thousand rows at automatic DOP, paste the `[run] snapshot` and `[timing] batch` lines.
- **Throughput effect:** `neutral`.
- **Deviation from plan:** none found (text matches Task 10).
- **Notes:** No live-smoke evidence appears anywhere in the window, and no tag was created.

`194345f` · **2026-11-03 01:17 -0500 · Clarify live write smoke evidence and environment**

- **Plan task / recommendation:** Task 10 follow-up.
- **Area:** docs
- **Files:** `documentation/how-to/developer/diagnostics.md`
- **Before → After:** Requires a "non-production" environment. Softens step 5: attempts above 1 with a total in minutes "is a signal to inspect", and "does not prove a sleep inside one attempt".
- **Throughput effect:** `neutral`.
- **Deviation from plan:** Yes, wording differs from the plan's text.
- **Notes:** none.

`a5fbbe5` · **2026-11-03 01:25 -0500 · Prevent ambiguous bulk create retries**

- **Plan task / recommendation:** none (beyond the plan); extends recommendation 3 from timeouts to network faults.
- **Area:** retry, timeout+cancel, batch size, progress+logging
- **Files:** `src/SeedBomb.Bulk/ThrottlePolicy.cs`, `src/SeedBomb.Bulk/BulkCreator.cs`, `src/SeedBomb.Wpf/ViewModels/RunViewModel.cs`, `src/SeedBomb.Bulk.Tests/BulkCreatorTests.cs`, `src/SeedBomb.Bulk.Tests/ThrottlePolicyTests.cs`, `src/SeedBomb.Wpf.Tests/RunViewModelTests.cs`
- **Before → After:**
  - `ExecuteAsync(..., bool resendOnAmbiguousTransport = true)`. The retry filter adds `&& (resendOnAmbiguousTransport || !IsAmbiguousTransport(ex))`.

    - New `public static bool IsAmbiguousTransport(Exception? ex) => ex is HttpRequestException or IOException || ex is CommunicationException c && IsWrappedNetworkFault(c);`
    - Both `CreateMultiple` and `ExecuteMultiple` now pass `resendOnTimeout: false, resendOnAmbiguousTransport: false`.
    - **Retries now prevented for create batches:** `HttpRequestException`, `IOException`, and `CommunicationException` wrapping either (for example socket reset 10054). Before this commit these were resent up to 3 times with 2·2^n s ± 2 s backoff.
    - **What happens instead:** `IsClientTimeout` is renamed `IsAmbiguousWriteOutcome` and also matches these exceptions. The batch then calls `trace.MarkTimeout()` and `sizer.ObserveTimeout()` (halving the table's shared batch size) and goes to `ResolveUnknownOutcomeBatchAsync` (renamed). A keyed batch is split into sequential halves, recursively; an unkeyed one gets a `BatchError` "The {Type} failed after {s} s; outcome unknown...", `OutcomeUnknown = true`, not retryable.
    - Still retried for creates: `EndpointNotFoundException`, `ServerTooBusyException`, 429/502/503 `ProtocolException`, and service-protection faults.
    - Reads, backfill and associate are unchanged (they still resend).
    - `RunViewModel`: a "Waiting" snapshot that arrives after the headline is "Cancelled" is ignored.
- **Throughput effect:** `lowers` in the degraded (network-flaky) state.
    - A transient connection reset or refused connection on any create batch is never retried. On unkeyed tables those rows are not written by this run. Each such fault also halves the table's batch size.
    - `HttpRequestException` also covers failures before any byte reached the server (DNS, connection refused), which are not actually ambiguous. Treating them as unknown loses rows that a resend could have written safely.
    - Healthy: neutral.
- **Deviation from plan:** Yes; not in the plan. The plan's Task 5 deliberately kept network faults retryable for creates (the "After the plan" notes do not mention it).
- **Notes:** Tests:
    - `CreateAsync_AmbiguousCreateMultipleTransportFailure_IsUnknownWithoutResend`: an `HttpRequestException` is sent once, `OutcomeUnknown`, 10 rows.
    - `..._SplitsKeyedBatch`: a wrapped `CommunicationException` gives sizes `[10, 5, 5]`.
    - `CreateAsync_AmbiguousExecuteMultipleTransportFailure_IsUnknownWithoutResend`: 1 send.
    - `ExecuteAsync_AmbiguousTransportResendDisabled_DoesNotRetryNetworkButStillRetriesRejection`: network is 1 call and still `IsTransient`; 429 is retried, 2 calls.
    - `AcceptProgress_QueuedWaitingSnapshotAfterCancellation_IsIgnored`.

Parameter snapshot at a5fbbe5 (HEAD)

- **Batch size.**

  - Default 500 (`src/SeedBomb.Core/Contracts/GenerationLimits.cs:15`, `GenerationConfig.cs:35`); allowed 1–1000 (`GenerationLimits.cs:12`, validated at `:33`).
  - Adaptive per table (`src/SeedBomb.Bulk/AdaptiveBatchSize.cs:11-18`, created at `BulkCreator.cs:425`):
    - start `max(1, min(configured, 250))`;
    - grow `cur + cur/2`, capped at configured, after a call ≤ 20 s (`AdaptiveBatchSize.cs:28-33`);
    - shrink `max(min(25, configured), cur/2)` on every timed-out or network-faulted create call (`AdaptiveBatchSize.cs:36-40`; called at `BulkCreator.cs:572`, `:587`);
    - no growth for a batch that had one (`BulkCreator.cs:479`).
  - The configured size is the ceiling. The sizer is shared by the table's workers, locked, and reset for every table. Slices are taken from a shared cursor at the current size (`BulkCreator.cs:431-441`).
  - Backfill and associate still use the fixed `config.BatchSize` (`BulkCreator.cs:220`, `:229`).

- **DOP** (unchanged in this window; `BulkCreator.cs:858-883`):

  1. If DOP is automatic (`MaxParallelism` null) and rows are planned, a WhoAmI warmup runs first (`:143`, `:889`) so `x-ms-dop-hint` reaches `ServiceClient`.
  2. For each table, read `RecommendedDegreesOfParallelism` (`:851`).
  3. Configured DOP: use it if the hint is ≤ 0 or configured ≤ hint, otherwise use the hint and add a notice (`:863-872`).
  4. Automatic DOP: hint ≤ 0 gives `DefaultDop = 8` (`:32`, `:876`); otherwise the hint as given, even above 16 (`:882`).
  - Configured range 1–16, with 0 meaning automatic (`GenerateViewModel.cs:874`).

- **App retries.** `MaxRetries = 3`, so at most 4 attempts (`GenerationConfig.cs:46`).

  - Delay with `Retry-After`: `ErrorDetails["Retry-After"]` as a `TimeSpan` > 0, exact and uncapped (`ThrottlePolicy.cs:187-191`, `:89-90`).
  - Delay otherwise: `min(2 s·2^attempt, 60 s) + U(−2, +2) s` (`ThrottlePolicy.cs:204-210`), i.e. [0, 4) s, [2, 6) s and [6, 10) s for attempts 0 to 2. There is no floor, so a service-protection fault without a header can be resent almost at once.

- **Retried types** (`ThrottlePolicy.cs:158-175`): `TimeoutException`, `OperationCanceledException` not from the caller, `HttpRequestException`, `IOException`, `EndpointNotFoundException`, `ServerTooBusyException`, `FaultException<OrganizationServiceFault>` with code -2147015902, -2147015903 or -2147015898, `ProtocolException` containing "(429)", "(502)" or "(503)", and `CommunicationException` wrapping `HttpRequestException` or `IOException`.

  - For create batches (`BulkCreator.cs:560`, `:582`), timeouts and the network classes are **not** resent (`ThrottlePolicy.cs:86-87`).

- **ServiceClient settings** at construction (`DataverseConnectionService.cs:118-132`):

Client setting	Value	Fed by (`AppSettings.cs:30-32`, clamps `:43-45`)
`MaxRetryCount`	`SdkMaxRetryCount`	default 0, range 0–10
`RetryPauseTime`	`SdkRetryPauseSeconds`	default 0 s, range 0–300
static `MaxConnectionTimeout`	`ConnectionTimeoutMinutes`	default 4 min, range 1–30
`EnableAffinityCookie`	`false`	not a setting

Settings are read when a new client is built.

- **Fallback.** `ExecuteMultiple` is used only when `CreateMultiple` faults with `0x80040800`; the table is then marked unsupported (`BulkCreator.cs:562-568`, `:666-668`). Any other terminal fault records one `BatchError` for the whole batch (`:463-475`).
- **Timeout.** `ServiceClient.MaxConnectionTimeout`, default 4 min (`DataverseConnectionService.cs:120`). On a timeout or network fault, a create is never resent whole: a keyed table splits into sequential halves, recursively down to 1 row (`BulkCreator.cs:604-611`); an unkeyed table records `OutcomeUnknown` and does not resend (`:614-622`). The batch size halves in both cases.
- **Shared pause.** A service-protection fault with a `Retry-After`, on a non-final attempt, extends one per-run deadline (`ThrottlePolicy.cs:26`, `:91-92`, `:120-130`; extend only, never cleared). Every `ThrottlePolicy` call in the run (creates of every table, lookup reads, backfill, N:N) waits for it before each attempt, re-checking until it has passed, while holding its worker slot (`:74`, `:132-142`). Only the faulting call raises `Waiting`, shown as "Server busy, resuming in N s (entity)" (`BulkCreator.cs:88-98`).

Before/after table for the window

Parameter	`b0654bc`	`a5fbbe5`
Default batch size	500	500
Batch size used	fixed `config.BatchSize` chunks	adaptive: start `min(cfg, 250)`, ×1.5 after a call ≤ 20 s up to cfg, ÷2 (floor 25) per timeout or network fault, per table
Partitioning	`Chunk(BatchSize)` up front	shared cursor, slice sized when taken
DOP resolution	warmup; explicit capped at hint; else hint; else 8	unchanged
App retries	3 (4 attempts)	3 (4 attempts)
SDK `MaxRetryCount` / `RetryPauseTime`	not set (SDK default; investigation: 10 / 5 s)	0 / 0 (settings, defaults 0 / 0 s)
`MaxConnectionTimeout`	not set (SDK default; investigation: 4 min)	set from settings, default 4 min, range 1–30
Service-protection fault	terminal in the app (the SDK retried it)	retried by the app, waiting `Retry-After` or backoff

Parameter	`b0654bc`	`a5fbbe5`
502/503 `ProtocolException`	not retried	retried with backoff
Delay formula	`min(2·2^n, 60) s ± 2 s`	`Retry-After` uncapped when present, else `min(2·2^n, 60) s ± 2 s`
Timeout on a create	resent up to 3× (up to 4 attempts × timeout), then "Retryable"	not resent; keyed: recursive sequential split; unkeyed: `OutcomeUnknown`
Network fault on a create	resent up to 3×	not resent; same split or unknown handling as a timeout
Fallback condition	any `FaultException` from `CreateMultiple`, or `0x80040800`	only `0x80040800`
Data fault in a batch	`ExecuteMultiple` replay writes the good rows	whole batch rejected as one `BatchError`
Shared pause	none	one per-run deadline, extend-only, all policy callers wait before each attempt
UI wait notice	none	"Server busy, resuming in N s (entity)" per retry sleep
Lookup pre-reads	every `lookupRandom` target read (≤ 1,000 rows, 500/page)	skipped for implicit in-run columns; lazy fallback read if the parents wrote 0 rows
Per-batch failure log	`LogError` with exception	`LogWarning` message only, plus one summary per table and message
Activity list	every line appended	identical consecutive lines collapsed "(×N)"
User settings	none for the client	SDK retries, SDK retry pause, call timeout

5. Before the window: Sept 26 to 30

The owner's last fast build is from 9/27, so the days just before Oct 1 matter as well. 43 non-merge commits landed between `16c3e75` (Sept 27, 18:20 EDT) and the start of the window. Over `src/SeedBomb.Bulk`, `Services/Dataverse` and `Core/Contracts` their combined diff is 61 added and 18 removed lines, and one commit changes the write loop.

Date	Commit	Change	Effect on a run
Sept 26	`41963f6`	`MaxRetryCount = 0` and `RetryPauseTime = 0` removed from the `ServiceClient`, so the SDK default of 10 retries applies. This is already in `16c3e75`.	A throttled call waits and resends inside the SDK instead of failing at once. A 9/27 binary built from a checkout older than this commit would still fail fast, which looks fast.
Sept 28	`7ea027b`	Explicit parallelism clamped to 16	Only for a typed value above 16
Sept 28	`7e8a00b`	Connection shutdown no longer blocks on an in-flight connect	None

Date	Commit	Change	Effect on a run
Sept 28	`e5bc58b`	Edge-case categories become an enum	None (refactor; the only `BulkCreator.cs` change in these days)
Sept 29	`eccf60a`	**A cancel the caller did not request (an HttpClient timeout) is retried up to 3 times.** Before, it ended the run as cancelled.	The only write-loop change between a `16c3e75` build and v1.0.0. A batch that reaches the 4-minute timeout can now hold a worker for about 16 minutes.
Sept 29	`136fc9b`	A client that failed to connect is disposed and never cached	None
Sept 30	`80debdf`	The Blazor web app is removed	The Blazor host set `ThreadPool.SetMinThreads(100, 100)` and `ServicePointManager.DefaultConnectionLimit = 65000` (`src/SeedBomb.Web/Program.cs:19-21` at `16c3e75`). The WPF app never set either. If the fast 9/27 runs were made in the Blazor app, they came from a different host process.

6. Possible root causes of the throughput drop

6.1 What sets the rate

For one table, rows per second is roughly `W × B / L(B)`:

- `W` is the number of requests in flight: the degree of parallelism, or fewer when fewer batches remain.
- `B` is rows per request.
- `L(B)` is the server's time for one request. It grows roughly in step with `B`, so the rate depends mostly on `W` and only weakly on `B`.

Tables are written one after another, and each table's rows are generated on one thread before its first request. Both throughput figures in the UI count rows that were written: the live figure is written rows over minutes since the table's first request (`BulkCreator.cs:493-495`), and the summary figure is written rows over the whole run's elapsed time (`RunViewModel.cs:918`). A rejected batch therefore lowers the figure as much as a slow one.

With automatic parallelism and no throttling, the terms at each build are:

Build	`W`	`B`
9/27 and v1.0.0	8 for the first table written, the hint afterwards	500
v1.0.1, v1.0.2, master	the hint for every table	500
Branch	the hint for every table	250, then 375, then 500, restarting for each table

Dataverse's documented service-protection limits are per user and per web server: 6,000 requests, 1,200 seconds of combined execution time, and 52 concurrent requests, each over a 5-minute sliding window. Requests that are always busy accrue execution time at `W` seconds per second, so more than four of them against one web server pass 1,200 seconds within the window: after about 150 seconds at 8, and about 240 seconds at 5. With several web servers and the affinity cookie off (as here), the load spreads and the threshold rises in proportion. Trial environments have one web server. This is inference from the documented limits: a run that lasts more than a few minutes should expect execution-time throttling, and what the client does then decides most of the rate.

6.2 Which candidate fits which build

If the slow build is	In a healthy run	In a throttled or faulty run
v1.0.0	Nothing in the write loop differs from a `16c3e75` build	RC3: timeouts resent since `eccf60a`. The nested SDK retries and the any-fault fallback were already in a `16c3e75` build.
v1.0.1, v1.0.2 or master	RC1	RC3, with a wider app retry set since `256faa9` and `35f4d23`
The branch	RC1, and a small adaptive-size effect	RC2

RC4 and RC5 apply to every build.

6.3 Candidates, ranked

RC1. Automatic parallelism now follows the server hint on every table (v1.0.1 and later)

- **Mechanism.** At 9/27 and v1.0.0, `effectiveDop = config.MaxParallelism ?? 8`, and the hint replaced it only after the table at topological index 0 had finished (`BulkCreator.cs:95`, `:137-148` at `16c3e75`). A single-table run therefore ran at 8 throughout, and a multi-table run ran its first table at 8. Since `7cb61b5`, `ResolveDop` runs at the top of every table and returns the hint (`BulkCreator.cs:161`, `:746-759` at v1.0.2; `:186`, `:858-883` at HEAD). The hint is `ServiceClient.RecommendedDegreesOfParallelism`: the last `x-ms-dop-hint` the client saw, or the SDK's default of 5.
- **Why it ranks first.** It is the only change since 9/27 that alters the rate of a run in which nothing goes wrong. The owner ran with automatic parallelism, which is the case it affects.
- **Size.** `1 − hint/8` on the affected tables when the hint is below 8: 37% at 5, 50% at 4, 75% at 2. Nothing if the hint is 8. If the hint is above 8 the run is faster at first, with no clamp at 16, and reaches the execution-time limit sooner. In a multi-table run only the first table changes.
- **Confidence.** High on the mechanism. The size is unknown until the hint is read from a log.
- **Does not explain.** Slowness on v1.0.0, or on a multi-table run whose rows are mostly in later tables.
- **Confirm.** `[run] snapshot: … configured DOP auto, SDK DOP hint N` and `DOP N` on each `[timing] batch` line.

- **Test.** On v1.0.2 or master, type 8 into Parallelism and run the same profile; an explicit value wins there. On the branch the explicit value is capped at the hint (`837d63b`), so the test needs v1.0.2 or master.
- **Note.** Microsoft's guidance is to use the hint and not to exceed it. A lower rate at the hint can still be the better sustained rate over a long run. Whether 8 was really faster on this environment is what the test above answers.

RC2. On the branch, one throttle stops the whole run, and a throttled batch is dropped after four attempts

All of this is read at HEAD.

1. The SDK no longer retries anything: `SdkMaxRetryCount` defaults to 0 (`AppSettings.cs:30`, applied at `DataverseConnectionService.cs:130`).
2. A service-protection fault that carries `Retry-After` arms one deadline for the whole run (`ThrottlePolicy.cs:89-92`, `:120-130`). One `ThrottlePolicy` is built per run (`GenerationPipeline.cs:78`), so every call that goes through it waits on that deadline before each attempt, while holding its worker: creates for every table, lookup reads, backfill and N:N (`ThrottlePolicy.cs:74`, `:132-142`).
3. The deadline only ever extends. Calls that were already in flight when it was armed can come back throttled and push it further out, and waiters loop until it has passed (`a2c6bcb`). `Retry-After` is used as given, with no cap.
4. There is no jitter on a `Retry-After` wait, so every worker resumes in the same instant with a full batch. The 5-minute window slides, so the budget is still nearly spent at that moment. Inference: a simultaneous burst is the likeliest way to be throttled again at once.
5. A batch gets 3 retries (`GenerationConfig.cs:46`). After a fourth throttled attempt all its rows are rejected (`BulkCreator.cs:463-475`), marked retryable. On master the SDK alone waited and resent up to 10 times inside each app attempt.
6. On that fourth attempt the fault arms no pause, because the pause is armed only while `attempt < maxRetries` (`ThrottlePolicy.cs:85`). The other workers keep sending to a server that has just refused.
7. A service-protection fault without a `Retry-After` detail, and an HTTP 429 that arrives as a `ProtocolException`, use the generic backoff and arm no pause. The first wait is uniform between 0 and 4 seconds (`ThrottlePolicy.cs:90`, `:204-210`), so the resend can be almost immediate.
8. `ExecuteMultiple` no longer replays a faulted batch (`c8a8b0f`). `CreateMultiple` on a standard table is all or nothing, so one bad row rejects every row in the batch.
9. A timeout or network fault on a create is never resent (`5567cf6`, `0cfb873`, `a5fbbe5`). On a table without an alternate key the rows are reported as outcome unknown and are not written by this run. Each such event halves the table's shared batch size (`BulkCreator.cs:572`, `:587`), and concurrent events compound: five take 500 down to the 25-row floor, and it takes eight fast calls to grow back. `HttpRequestException` is treated as ambiguous even when the request never left the machine (connection refused, DNS).
10. Calls that do not go through `ThrottlePolicy` now have no retry at all: the currency and user pool reads (`BulkCreator.cs:796`, `:829`), the WhoAmI warmup (`:889`), metadata retrieval, the `UpdateMultiple` availability probe, and the watermark and reconcile queries.

- **Size.** A model, not a measurement: parallelism 4, 500-row batches taking 12 seconds, and one throttle a minute with a 30-second `Retry-After`. On master one worker sleeps and the other three carry on, a loss of about 12%. On the branch all four idle, a loss of 42 to 50%. On an environment with

one web server every request is refused during the wait in both builds, so the two are close. On top of that, rows lost to items 5, 8 and 9 come straight off the throughput figure.
- **Confidence.** High on the mechanism. Medium that it is material: the earlier symptom on this environment was timeouts and HTTP errors, which is what throttling looks like, but how often it throttles is not known.
- **Confirm.** `Service protection for {table} on attempt n of 4 … Retrying in ~N ms` warnings; "Server busy, resuming in N s" on the run sheet; `[timing] batch` lines with `1 attempt(s)` whose `total` is far above `last call` (a batch that was never throttled itself but waited on the shared deadline); `N batch(es) (M rows) failed with:` summaries.
- **Test without code.** Settings, "Advanced: Dataverse client", set SDK max retries to 10, reconnect (the setting applies to the next connection), and run the same profile. The SDK then absorbs each throttle inside the call as on master, and the shared pause rarely arms.

RC3. From v1.0.0 to master, a bad batch holds a worker for minutes

This is the earlier investigation's F1 to F3, restated as a throughput cost.

- **Mechanism.** The SDK retries up to 10 times inside each of the app's 4 attempts. A timeout is resent 3 times at up to 4 minutes each (`eccf60a`, `ThrottlePolicy.cs:55-61` at v1.0.0), about 16 minutes of one worker, with duplicate rows if an earlier attempt had committed. Any fault from `CreateMultiple` replays the whole batch through `ExecuteMultiple` as single creates (`BulkCreator.cs:482` at v1.0.0, `:504` at v1.0.2). From v1.0.1 the app also retries wrapped socket resets, `EndpointNotFoundException` and `ServerTooBusyException`.
- **Size.** One worker out of `W` for the length of each stall, so a quarter of capacity at parallelism 4 for up to 16 minutes per timed-out batch. Nothing in a run with no timeouts, throttles or faults.
- **Against 9/27.** A build from `16c3e75` already had the SDK retries and the fallback. Only the timeout resend, and from v1.0.1 the wider retry set, are new. So RC3 explains a drop relative to 9/27 where timeouts are involved, or where the 9/27 binary predates `41963f6`.
- **Confidence.** Medium for stalls, low for a steady drop in rate.
- **Confirm (v1.0.1 and later).** `Timeout for … on attempt n of 4 after ~240000ms`; `[timing] batch` with `last call` near 240,000 ms; `1 attempt(s)` with a `last call` far longer than its neighbours (the SDK sleeping inside the call); `via CreateMultiple>ExecuteMultiple`. v1.0.0 has no `[timing]` lines; only the retry warnings.

RC4. Costs that predate 9/27 and sit inside the throughput figure

Not regressions, so they cannot explain a drop on an unchanged profile. They matter if the profile grew or if the summary figure is the one being compared.

- **Serial generation.** Every row of a table is built on one thread before the table's first request (`BulkCreator.cs:321-383`), and tables do not overlap. The network is idle meanwhile. Estimated at 0.1 to 0.5 ms per row for wide tables.

- **Backfill.** Deferred lookups (any profile with a cycle, such as account and contact) are updated after all creates, one `UpdateMultiple` batch at a time (`DeferredLookupBackfill.cs:265-301`). Building the updates calls `pool.Get(target)` once per source row (`DeferredLookupBackfill.cs:120`), and `Get` copies the whole target list each time (`DataverseRecordPool.cs:45`). That is 5,000 copies of a 20,000-item list for 5,000 accounts pointing at 20,000 contacts. The backfill for a table can take as long as creating it.
- **N:N.** One `AssociateRequest` per row, in `ExecuteMultiple` batches sent one at a time.
- **Confirm.** `[timing] phase generate`, `phase submit`, `phase backfill` and `phase associate` (v1.0.1 and later) show where the wall-clock time went.

RC5. Conditional: a per-row warning when the currency pool is empty

- **Mechanism.** If the `transactioncurrency` read fails or returns nothing, every row of every table with a money column logs a warning inside the generation loop (`BulkCreator.cs:374`). Each warning is a synchronous file append under one global lock (`FileLoggerProvider.cs:44-48`). The warning predates 9/27. The read is not retried by `ThrottlePolicy`, and on the branch the SDK no longer retries it either, so one transient fault at run start is enough there.
- **Size.** Estimated at 0.1 to 2 ms per row, so seconds to about a minute for tens of thousands of rows. Serial, before the table's first write.
- **Confidence.** High on the mechanism, low that it happened.
- **Confirm.** Count `No transactioncurrency in pool` lines in the log.

6.4 Checked and not implicated

- **Batch-loop locking.** The cursor lock and the size lock guard arithmetic only. Nothing is awaited under a lock, and workers do not serialise each other.
- **Adaptive batch size in a healthy run.** A simulation of the code's rules gives −10% to +9% against fixed 500-row batches. Growth is driven by the count of fast completions, so at parallelism 2 or more the size reaches the ceiling after the first few 250-row calls.
- **Jitter and backoff.** `ComputeDelay` is the same at every build: `min(60 s, 2 s × 2^attempt)` plus uniform −2 to +2 seconds. It runs only on error paths.
- **Connections.** One `ServiceClient` at every build, never cloned, affinity cookie off, no connection-limit or thread-pool settings in the WPF app. The SDK's async path takes no lock.
- **Tokens.** MSAL silent acquisition per request with no app-level lock, unchanged since August.
- **Progress and UI.** `Progress<T>` posts and never blocks a worker; the 100 ms gate predates 9/27; the activity list is capped at 200 lines.
- **Per-batch logging.** One line per batch since v1.0.1, milliseconds against requests that take seconds.
- **Session gate.** Acquired once per run and never on the token or write path.
- **Metadata.** A 30-minute cache at every build; a miss costs one parallel fetch per table at run start.
- **Pre-run lookup reads.** Added for implicit required lookups at v1.0.0 and removed again on the branch. Fixed seconds per run.
- **Payload.** No new per-row columns after v1.0.0; master removed `overriddencreatedon`.
- **Packages and platform.** `Dataverse.Client` 1.2.27 throughout. `Caching.Memory` and `Cryptography.Xml` patch bumps do not touch the write path. The shipped exe has been x64, self-contained, single-file, without ReadyToRun since before 9/27. The open Dependabot branches do not touch

`Dataverse.Client` and are in no build.
- **Diagnostics.** Run diagnostics is off by default. When on it adds one read per table before the first write and one count per table after the last.

6.5 Outside the code

The code cannot rule these out, and with no measurements they stay open:

- **The environment.** Its hint, the number of web servers behind it (a trial has one), plug-ins or flows on the target tables, and other load from the same user in the same 5-minute window.
- **The profile.** More tables, more lookups or a cycle add generation and backfill time that the summary figure includes.
- **The binary.** Whether the fast 9/27 runs came from the WPF app or the Blazor app, and which commit that build was made from.

7. What to capture to settle it

The log is `%LOCALAPPDATA%\SeedBomb\logs\seedbomb-yyyyMMdd.log`. Logs older than seven days are pruned, so copy it soon after the run.

Which build produced a log

- No `[timing]` or `[run]` lines: v1.0.0 or earlier.
- `[run] snapshot: app 1.0.x`: a tagged release.
- `app 2.0.1`: a local build of master or the branch (`Directory.Build.props`).
- A snapshot with `(starting at 250)` and `SDK retries 0`: the branch.

What to look for

In the log	Points to
`SDK DOP hint N` with N below 8, and `DOP N` on the batch lines	RC1
`Service protection for … Retrying in ~N ms`, or "Server busy" on the run sheet	Throttling. Count them and add up N.
`[timing] batch` with `1 attempt(s)` and `total` far above `last call`	Branch: the batch waited on the shared pause (RC2)
`[timing] batch` with `1 attempt(s)` and a `last call` far longer than its neighbours	master: the SDK slept inside the call (RC3)
`last call` near 240,000 ms	A client timeout: resent on master (RC3), split or reported unknown on the branch (RC2)
`via CreateMultiple>ExecuteMultiple`	master: a faulted batch replayed as single creates (RC3)
`failed` with `fault 0x…`, and `N batch(es) (M rows) failed with:`	Branch: whole batches rejected (RC2, items 5 and 8)

In the log	Points to
Row counts on batch lines falling to 125, 62, 25	Branch: the batch size collapsing after timeouts or network faults (RC2, item 9)
`[timing] phase generate`, `submit`, `backfill`	Where the wall-clock time went (RC4)
Many `No transactioncurrency in pool` lines	RC5

Runs worth making, on a non-production environment with the same profile each time:

1. One run on the build that feels slow, log kept. This alone separates RC1 from RC2 and RC3.
2. On v1.0.2 or master: Parallelism 8 against automatic (RC1).
3. On the branch: SDK max retries 10 against 0 (RC2).
4. If a 9/27 build still exists: the same profile on it, timed with a stopwatch, since that build writes no timing lines. This is the only way to put a number on the drop.

8. Sources

- This repository's history, `a527963..a5fbbe5`, and the Sept 26 to 30 commits before it.
- `sb-investigate.md` and `docs/plans/11-03-2026-write-path-retry-ownership.md`. Corrections to the first are in section 4.2 and in the notes of individual entries.
- Microsoft Learn: Service protection API limits, Send parallel requests, Use bulk operation messages, Optimize performance for bulk operations.
- The SDK's public source, `microsoft/PowerPlatform-DataverseServiceClient`, master branch: `ConnectionService.cs`, `ServiceClient.cs`, `ConfigurationOptions.cs`, `DataverseTraceLogger.cs`.